

claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-ab90acd72923cd2a5.jsonl`
- SHA-256: `58ec9c64dbf6eb6074455cda8d6253c7411d3aeac59b3387a72c738496591272`
- Source modified: `2026-06-07T09:17:12+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`
```

Transcript

1. user (2026-06-07T09:12:48.072Z)

You are regenerating docs/CODE_MAP.md for C:/Users/avidu/Projects/badminton-highlight-indexer after a directory restructure (#40:
backend/pipeline/{validators,segmenters,stitcher,detectors,audio}/,
backend/infrastructure/database.py, backend/api/static/, backend/config/) plus two NEW subsystems: backend/config/ (typed Pydantic AppConfig + loader; note AppConfig.get is a temporary dict-compatible shim that returns nested sections as dicts) and backend/reporting/ (per-video observability via the external soft-dependency 'obsreport' ? harvest.py + spec.yaml footgun rules; emitted from a finally block in backend/main.py; writes <video>.report.json/.report.md/.summary.md).

First READ the EXISTING map at C:/Users/avidu/Projects/badminton-highlight-indexer/docs/CODE_MAP.md to match its exact section structure, legend, and density. Then, using the freshly-mapped subsystem chunks below, output the COMPLETE updated CODE_MAP.md.
Requirements: (1) every @path reflects the CURRENT layout (no backend/validators, backend/indexer, backend/database, backend/stitcher, backend/detectors, backend/audio ? those moved). (2) Add subsystem sections for backend/config/ and backend/reporting/. (3) \$1 pipeline: show typed AppConfig load + the observability emit (never raises). (4) Refresh \$8 test map with the real current test files. (5) Keep \$4 (WASB WSL) and \$5 (cross-repo collector) but fix paths. (6) Preserve/extend \$6 gotchas and \$7 ramp paths with corrected paths + new entries (reporting, config, run_all_tests.py). Output ONLY the markdown, beginning with "# CODE_MAP.md".

=== SUBSYSTEM CHUNKS ===

chunk: Orchestration & config

I have all the files. Producing the dense CODE_MAP chunk now.

Subsystem: Orchestration & config

Entry points (CLI/API/scripts), the typed config layer, and the eval/regression/stitch drivers. All paths repo-relative.

Typed config layer (`backend/config/`)

```
- **@backend/config/__init__.py** ? public config API. Re-exports `load_config`,  
  `save_default_config` (from `.loader`) and `AppConfig`, `Config`, `IndexingConfig`,  
  `ValidationConfig` (from `.models`). `__all__` lists exactly these 6. ? Canonical import  
  surface: `from backend.config import load_config, AppConfig`.  
  
- **@backend/config/models.py** ? Pydantic v2 config models; single source of truth for  
  defaults.  
  - $@backend/config/models.py::Sport` ?
```

```

Literal["badminton","tennis","pickleball","table_tennis","cricket"]`.
- S`@backend/config/models.py::ValidationRelevanceConfig` ? court HSV gates:
`court_green_lower=[35,40,40]`, `court_green_upper=[85,255,255]`, `court_pixels_min_ratio=0.05`
(clamped `ge=0.0,le=1.0`).
- S`@backend/config/models.py::ValidationConfig` ? `min_resolution_width=1280`,
`min_resolution_height=720`, `min_fps=29.9`, `min_blur_threshold=20.0`,
`max_scene_cuts_per_minute=0.5`, `relevance` (nested).
- S`@backend/config/models.py::TrackNetConfig` ? WSL2/conda invocation contract:
`wsl_distro="Ubuntu"`, `repo_dir=~/.models/TrackNetV4`, `conda_sh`, `conda_env="TrackNetV4",
`weights_path=""`, `queue_length=5`, `stage_in_wsl=True`, `wsl_stage_dir=~/.clips`,
`timeout_sec=0`, `velocity_threshold=0.02`.
- S`@backend/config/models.py::IndexingConfig` ? segmenter/detector knobs. ?
**default_segmenter="yolo_hybrid" here, but the on-disk `config.json` overrides it to
`"gemini"` (see config.json gotcha). Other fields: `use_gpu`, `motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `detector_model="fasterrcnn_resnet50_fpn",
`detector_score_threshold=0.5`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`yolo_tracker="bytetrack.yaml", `yolo_parallel_chunks=False`, `yolo_chunk_workers=2`,
`min_action_window_duration=2.0`, `log_yolo_candidates=True`, `store_yolo_candidates=True`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, nested `tracknet`.
- ? `@backend/config/models.py::IndexingConfig.segmenter_must_be_registered` ?
`@field_validator("default_segmenter")`; intentionally a **no-op pass-through** (allows any
string for forward-compat; registry check deferred to runtime in main/segmenter selection). ? A
typo'd segmenter name will validate fine here and only fail later at
`segmenter_registry.get(...)` .
- S`@backend/config/models.py::OutputConfig` ? `default_highlights_name="highlights.mp4",
`db_name="sports_indexer.db"`. ? No `output_dir` field ? output dir is a **runtime-only**
concern (see main.py `_runtime_overrides`).
- S`@backend/config/models.py::StitchingConfig` ? `begin_padding=0.5`, `end_padding=0.5`,
`use_cache=False`, `min_shot_count=1`.
- S`@backend/config/models.py::AppConfig` ? root model. Scalars: `sport="badminton",
`ai_provider="gemini", `ai_model="gemini-2.5-flash"; sections `validation`, `indexing`,
`output`, `stitching` (all `default_factory`). `model_config =
{"extra":"allow","validate_assignment":True}`.
- ? `extra="allow" ? unknown keys in config.json are silently kept (legacy-migration
tolerance), so typos in section/key names are NOT rejected.
- ? `validate_assignment=True` ? `global_config.runtime_overrides = {}` in main.py works
only because `extra="allow" permits the dynamic attr; assignment is still validated.
- ? `@backend/config/models.py::AppConfig.get` ? legacy dict-style shim. Returns nested
**sections as plain dicts** via `model_dump(mode="json")` (NOT the Pydantic model) so chained
`config.get("indexing",{}).get(...)` keeps working; scalar leaves returned as-is. Falls back to
searching a leaf key inside any nested section (sections discovered dynamically from
`model_fields`, so a future top-level section is covered without code edits). ? This dict-vs-
model footgun "bit every validator on real runs" per the docstring ? returning the model breaks
`.get`.
- ? `@backend/config/models.py::AppConfig.__getitem__` ? `config[key]`; raises `KeyError`
for unknown top-level keys (dict-like), else delegates to `.get`.
- S`@backend/config/models.py::Config` ? alias `Config = AppConfig` (annotation convenience).

- **@backend/config/loader.py** ? load/save with defaults-merge precedence.
- `@backend/config/loader.py::DEFAULT_CONFIG_PATH` = `Path("config.json")` (CWD-relative). ?
Loader resolves config relative to the **current working directory**, not the repo root.
- `@backend/config/loader.py::get_builtin_defaults` ? `AppConfig().model_dump(mode="json")`;
models are the single source of truth.
- `@backend/config/loader.py::load_config` ? precedence: built-in defaults ? file overrides ?
(future) env/CLI. ? **Shallow top-level merge only** (`{**defaults, **file_data}`): if
config.json supplies a section (e.g. `"indexing"`), the entire file section REPLACES the
defaults dict for that section before validation ? partial sub-keys are not deep-merged with
model defaults at this layer (Pydantic then re-applies field defaults for any missing leaf).
Missing/unreadable file ? fully-defaulted `AppConfig` (parse errors print `[CONFIG WARNING]` and
continue, non-fatal).
- `@backend/config/loader.py::save_default_config` ? writes fresh `config.json`
(`model_dump_json(indent=4)`); **overwrites** if present. Used by setup.py / `--setup`.
- `@backend/config/loader.py::get_default_config` ? ? transitional alias returning defaults as
plain dict; "for transitional use only".

```

- `$**@config.json**` ? on-disk config; structurally mirrors ``AppConfig``. ?
- `**`indexing.default_segmenter="gemini"```` ? diverges from the model default ``"yolo_hybrid"````; the file wins at runtime. Note ``chunk_duration: 90`` / ``chunk_overlap: 10`` are JSON ints, coerced to float by Pydantic.

App entry point (``backend/main.py``)

- `**@backend/main.py::app**` ? ``FastAPI(title="Sports Highlight Indexer API")``; CORS allow-all; mounts ``/static`` from ``backend/api/static`` (dir auto-created).
- ``@backend/main.py::db_instance`` / ``@backend/main.py::global_config`` ? module-level globals (``Optional[Database]`` / ``Optional[AppConfig]``); initialized in ``main()``. ? API handlers assume these are non-None ? calling endpoints without going through ``main()`` (the only initializer) would NPE.
- ``@backend/main.py::get_file_md5`` ? 64 KB-chunked MD5 ? ``video_id``. ? Three copies of this exist with `**different chunk sizes**`: `main.py` 65536, `run_eval.py` 8192, `run_phase3_eval.py` 65536, `run_process_and_stitch.py` 1 MB. All produce the same hash (chunk size doesn't affect MD5), but it is duplicated, not shared.
- ``@backend/main.py::load_config`` ? ? legacy thin wrapper returning a `**dict**` (``load_app_config(path).model_dump(mode="json")``); kept for transition. New code should import ``load_app_config`` / ``AppConfig`` directly.
- `$`@backend/main.py::ProcessRequest`` ? ``video_path``, ``player_pool?``, ``max_frames?``, ``segmenter?``.
- `$`@backend/main.py::QueryRequest`` ? ``video_id``, ``server?``, ``receiver?``, ``duration_min?``, ``event_type?``.
- `$`@backend/main.py::CompileRequest`` ? ``video_id``, ``segment_ids: List[int]``, ``output_filename``.
- ``@backend/main.py::get_config`` ? ``GET /api/config`` ? returns ``global_config``.
- ``@backend/main.py::process_video`` ? ``POST /api/process``. Flow: md5 ? ``registry.run_all_with_context(path, global_config)`` ? (validator registry) ? on failure persist ``"failed"```` and return early ? else pick segmenter via ``req.segmenter`` or `global_config.indexing.default_segmenter`` (fallback ``"motion"````) ? ``segmenter_registry.get(seg_name)`` ? (segmenter registry; 400 with available list if missing) ? ``segmenter_cls(db_instance, global_config).process_video(...)``. ? ``finally:`` always calls ``emit_indexer_report(...)`` (never raises) and grafts ``response["reports"]`` ? report emission is unconditional even on validation failure / exceptions.
- ``@backend/main.py::query_play_segments`` ? ``POST /api/query``; builds filter dict and calls ``db_instance.query_play_segments``.
- ``@backend/main.py::compile_highlights`` ? ``POST /api/compile``. ? Reads output dir via ``getattr(getattr(global_config, "output", None), "output_dir", "output")`` or `"output"```` ? but ``OutputConfig`` has `**no `output_dir` field**`, so this always falls back to ``"output"```` (the ``_runtime_overrides["output_dir"]`` set in ``main()`` is NOT consulted here). Builds ``VideoCompiler(output_dir)``; wraps stitch in try/except ? 500.
- ``@backend/main.py::run_cli_diagnose_clip(video_path, timestamp)`` ? ``--debug-clip`` diagnostics: runs ``registry.run_all``, samples ± 3 s motion via OpenCV, compares avg motion to threshold. ? Uses legacy ``load_config()`` (dict path) and has a dual-path read: ``if hasattr(cfg, "indexing")`` (model) else ``cfg.get("indexing", {})``.get("motion_threshold", 0.08)` (dict) ? defensive because the wrapper currently returns a dict.
- ``@backend/main.py::main`` ? argparse orchestrator. Flags: ``--video-path``, ``--output-dir`` (default ``output``), ``--setup``, ``--debug-clip`` (float), ``--server``, ``--port`` (8000), ``--max-frames``, ``--segmenter`` (? ``choices`` populated dynamically from ``segmenter_registry.list_available()`` at parse time), ``--trim-start/--trim-end/--trim-output``. Modes (priority order): ``--setup`` shells out to ``setup.py`` via subprocess ? trim mode (``VideoCompiler.compile_highlights`` single pair) ? ``--debug-clip`` ? otherwise init ``global_config=load_app_config()`` + ``Database``, then CLI-process mode (video-path & not server) ? else server mode (``uvicorn.run``).
- ? ``@backend/main.py`` runtime-override hack: ``global_config._runtime_overrides = {"output_dir": args.output_dir}`` ? output dir lives only on this dynamic dict, NOT in any config model; downstream ``db_path = os.path.join(args.output_dir, output.db_name)``.
- ? CLI-process path mirrors the API ``finally:`` report-emission contract (always emits via ``emit_indexer_report``, prints ``technical_md`` / ``customer_md`` / ``json`` paths).

Standalone orchestration drivers (repo-root scripts)

- `**@run_eval.py**` ? CLI: score detected rallies in the DB vs hand-annotated GT (no pipeline run, deterministic). ? Does NOT run segmentation ? predictions must already be in the DB (epilog ``_EPILOG`` warns about this).

```

- `@run_eval.py::get_file_md5` (8 KB chunks), `@run_eval.py::default_db_path` (? reads
`config.json` **directly via json.load**, not `backend.config` ? bypasses the typed loader; ?
`output/<db_name>`), `@run_eval.py::EPILOG`, `@run_eval.py::build_parser`,
`@run_eval.py::main`.
- Flags: `--annotations`, `--video/--video-id`, `--db`, `--stage{candidates,final,both}`,
`--detector`, `--iou(0.5)`, `--pr-curve`, `--show-failures`, `--json`, `--scaffold`. Calls
`backend.eval.annotations.{load_annotations,write_scaffold}` and
`backend.eval.harness.{evaluate,format_report_text,report_to_dict,STAGES}`. Returns exit code
int.
- **@run_phase3_eval.py** ? Phase 3 batch driver: process+evaluate a collector `manifest.json`
export, per-video + corpus aggregate.
- `load_dotenv()` at import (GEMINI_API_KEY for AI segmenters).
`@run_phase3_eval.py::get_file_md5` (64 KB), `@run_phase3_eval.py::load_config` (? **own**
json.load-based dict loader, not `backend.config`), `@run_phase3_eval.py::build_parser`,
`@run_phase3_eval.py::main`.
- Flags: `--manifest` (required), `--videos-root` (default two levels above manifest),
`--segmenter` (default `motion`), `--stage{candidates,final,both,auto}` (default `auto`),
`--detector`, `--iou`, `--only` (repeatable), `--limit`, `--max-frames`, `--score-only`,
`--reprocess`, `--db`, `--json`, `--show-failures`.
- $Manifest contract: top-level `eval_sets[]`, each entry `{video_id, local_path, csv}`. ?
`segmenter_registry.get(...)` resolution; `segmenter_cls(db, cfg)` where **cfg is a raw dict**
(not AppConfig). Uses
`backend.eval.batch.{default_stages_for,resolve_video_path,aggregate,format_aggregate}` +
harness. ? Skip-rather-than-fail per entry (missing video/CSV ? `skipped`); idempotency via
existing `query_play_segments`/`query_candidate_segments` unless `--reprocess`.
- **@run_regression.py** ? CLI: regression-gate the tool vs golden-set GT; **exit 1 on
regression** (CI gate).
- `@run_regression.py::default_db_path` (? again direct json.load of config.json),
`@run_regression.py::build_parser`, `@run_regression.py::main`.
- Flags: `--video-id` (required), `--tier` (default `file`; ? help lists
`annotation_registry.list_available()`), `--annotations` (video_ref for provider),
`--stage{candidates,final}`, `--detector`, `--iou(0.5)`,
`--tolerance` (`regression.DEFAULT_TOLERANCE`), `--db`,
`--baseline` (`regression.DEFAULT_BASELINE_PATH`), `--update-baseline`, `--json`. ?
`backend.annotations.annotation_registry` (provider-tier plugin point). Calls `backend.eval.regr
ession.{regress_with_provider,format_result,save_baseline,DEFAULT_TOLERANCE,DEFAULT_BASELINE_PAT
H}`; result fields `.video_id/.stage/.precision/.recall/.f1/.regressed/.as_dict()`. ? `return 1
if result.regressed else 0`.
- **@run_process_and_stitch.py** ? end-to-end one-shot: process a hardcoded video ? segment ?
stitch highlights.
- `@run_process_and_stitch.py::get_file_md5` (1 MB chunks, prints GB progress),
`@run_process_and_stitch.py::main`.
- ? **Hardcoded paths** in `main()`:
`video_path=C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4`,
`output_dir=...\Testing`, `output_filename="Final_2_OutputOf..."`. Not parameterized ? edit-to-
run script.
- ? Config read via raw `json.load` (dict), NOT `backend.config`. ? `use_cache` default here
is `True` (`stitching.get("use_cache",True)`) ? opposite of `StitchingConfig.use_cache=False`
and config.json's `false`.
- ? **Interactive `input()` prompts** (cache reuse 1/2; begin/end padding + min_shot_count
overrides) ? blocks on stdin, unsuitable for headless/CI. ? `end_padding` default fallback `1.0`
here vs config/model `0.5`; `min_shot_count` fallback `0` here vs config/model `1`.
- Filters segments by `metadata.shot_count >= min_shot_count` (handles `metadata` as dict OR
JSON string). ? `segmenter_registry.get(seg_name)` with `seg_name =
config.get("indexing",{ }).get("default_segmenter","yolo_hybrid")` (? literal fallback
`yolo_hybrid` differs from config.json's `gemini`). `VideoCompiler(output_dir, end_padding=,
begin_padding=)`.

```

Setup / bootstrap

```

- **@setup.py** ? dependency checker + config initializer; run directly or via `main.py
--setup`.
- `@setup.py::check_command` (`shutil.which`), `@setup.py::run_pip_install` (? GPU-aware:
`nvidia-smi` present ? CUDA wheels `cu124`, else CPU wheels; `pip install --user -r
requirements.txt`), `@setup.py::init_default_config` (? delegates to

```

```
`backend.config.save_default_config` ? single-source defaults; **skips if config.json exists**,
so it won't clobber), `@setup.py::run_diagnostics` (Python ?3.8, ffmpeg/ffprobe in PATH ?
flagged CRITICAL/REQUIRED, optional torch+CUDA probe).
- `__main__`: `init_default_config()` ? `run_diagnostics()` ? `run_pip_install()`.
```

Cross-cutting gotchas (orchestration-wide)

```
- ? **Two config-access idioms coexist.** `backend/main.py` uses the typed `AppConfig`
(attribute access + `getattr` fallbacks); all four `run_*.py` scripts read `config.json` via raw
`json.load` into a dict and bypass `backend.config` entirely. Default-drift between the model,
`config.json`, and per-script literal fallbacks is the recurring footgun (segmenter, paddings,
`min_shot_count`, `use_cache`).
- ? **Segmenter default is defined in 5 places with 3 different values:** model `yolo_hybrid`,
config.json `gemini`, main.py fallback `motion`, phase3 default `motion`, process_and_stitch
fallback `yolo_hybrid`.
- ? **Three runtime registries** are the orchestration extension points: validator `registry`
(`backend.pipeline.validators.base`), `segmenter_registry`
(`backend.pipeline.segmenters[.base]`, `.get`/`.list_available`), and `annotation_registry`
(`backend.annotations`, `.list_available`).
- ? **--segmenter CLI `choices` in `main.py` are frozen at argparse-build time from the registry
? a segmenter registered later (lazy import) wouldn't appear.
```

Note on `tests/`

The task listed `tests/` for enumeration, but no `tests/` directory was among the provided paths and none of the read files reference one. No test files were inspected; mapping omitted to avoid fabrication. (If a test suite exists, it should be enumerated in a follow-up pass against the actual directory.)

chunk: Validators

Validators subsystem

Layered validation pipeline (`backend/pipeline/validators/`): each validator is a plugin implementing the `VideoValidator` ABC, registered into a global ordered `registry`, and run in sequence against an MP4 with a shared mutable `context` dict (used to pass ffprobe metadata downstream). All validators return a uniform `ValidationResult`.

base ? contracts + registry

```
- $ `ValidationResult` @backend/pipeline/validators/base.py::ValidationResult ? pydantic
`BaseModel`; the uniform output contract. Fields: `validator_name: str`, `passed: bool`,
`message: str`, `details: Dict[str, Any] = {}`. ? `details` default `{}` is a shared mutable
class-level default (Pydantic copies per-instance so safe here, but note it's the contract every
validator populates).
- ? `VideoValidator` @backend/pipeline/validators/base.py::VideoValidator ? ABC plugin base.
Abstract `@property name -> str` (unique id), abstract `@property description -> str`, abstract
`validate(self, video_path: str, config: Dict, context: Dict) -> ValidationResult`. $`validate`
signature is THE plugin contract: `config` is parsed `config.json`; `context` is the shared per-
run dict.
- ? `ValidationRegistry` @backend/pipeline/validators/base.py::ValidationRegistry ? ordered
registry (`self._validators: List[VideoValidator]`, a list not a map ? registration ORDER is
execution order).
- `register(validator)` @backend/pipeline/validators/base.py::ValidationRegistry.register ?
appends; ? no dedup / no name-collision check, duplicates run twice.
- $`run_all_with_context(video_path, config) -> (results, context)`
@backend/pipeline/validators/base.py::ValidationRegistry.run_all_with_context ? runs every
validator in sequence, accumulating `context`; wraps each `validate` in try/except so one
validator's exception becomes a failed `ValidationResult` instead of aborting the run. ? the
exception-path `ValidationResult` omits `message`?? no ? sets `message` but NOT `details`; uses
`val.name`. Callers wanting media metadata read the returned `context['ffprobe_meta']` to avoid
re-probing.
- `run_all(video_path, config) -> List[ValidationResult]`
@backend/pipeline/validators/base.py::ValidationRegistry.run_all ? thin wrapper that discards
`context`.
- ? `registry` @backend/pipeline/validators/base.py::registry ? module-level singleton
```

`ValidationRegistry()`; the global extension point.

- ? @backend/pipeline/validators/__init__.py ? import-time side-effect wiring: instantiates and `registry.register(...)`s the 6 default validators in dependency order: `FormatValidator` ? `ResolutionFPSValidator` ? `PTSContinuityValidator` ? `SceneCutDensityValidator` ? `BlurValidator` ? `RelevanceValidator`. Re-exports `registry`, `VideoValidator`, `ValidationResult` via `\_\_all\_\_`. ? ORDER MATTERS ? `FormatValidator` must run first because it populates `context['ffprobe\_meta']` that `ResolutionFPSValidator` depends on. ? importing this module mutates the global `registry` as a side effect; importing twice would re-register (idempotency relies on Python module caching).

Validators (each implements VideoValidator)

- `FormatValidator` @backend/pipeline/validators/format.py::FormatValidator ? name
`"format\_check"`. Runs `ffprobe -show\_format -show\_streams -of json` on `v:0`. ? PRODUCER:
stores full probe JSON into `context["ffprobe\_meta"]` for downstream validators. Returns
`details={format\_name, duration(float), size\_bytes(int), codec\_name}`. Fails on: missing file
(`os.path.exists`), empty streams/format, `format\_name` not containing "mp4" (substring, case-
insensitive), `CalledProcessError` (corrupt/unreadable). ? MP4 check is a loose substring match
on `format\_name` (ffprobe reports e.g. "mov,mp4,m4a,3gp,3g2,mj2" so most MOV-family files
pass). ? external `ffprobe` binary dependency.

- `ResolutionFPSValidator`
@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator ? name
`"resolution\_fps\_check"`. CONSUMER: reads `context.get("ffprobe\_meta")`; fails fast with "Run
format_check validator first" if absent. Parses `width`/`height` and `r\_frame\_rate` ("num/den"
rational). \$config keys under `config["validation"]`: `min\_resolution\_width` (default 1280),
`min\_resolution\_height` (default 720), `min\_fps` (default 29.9). `details={width,height,fps}`. ?
depends on FormatValidator having run first (ordering coupling via context). ? `fps` parse
catches only `ValueError` ? 0.0; a non-"a/b" string lacking `/` raises elsewhere but split
form is assumed.

- `PTSContinuityValidator`
@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator ? name
`"pts\_continuity\_check"`. Independently shells `ffprobe -show_entries packet=pts_time,flags -of
csv=p=0` (does NOT use `context`; re-probes). Keeps only keyframe packets ("K" in flags),
computes `max\_keyframe\_gap\_seconds`, `backward\_jumps\_detected`. Fails on: any backward jump, or
`max\_gap > 10.0`s (? hard-coded threshold, NOT config-driven ? comment mentions 8.0s but code
uses 10.0). PASSES (not fails) if `< 2` keyframes ("too short"). `details={keyframes_count,max_k
eyframe_gap_seconds,backward_jumps_detected,first_keyframe,last_keyframe}`. ? keyframe gap
analysis assumes raw continuous capture (GOP 174s).

- `SceneCutDensityValidator` @backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator
? name `"scene\_cut\_check"`. OpenCV (`cv2.VideoCapture`). Samples ~2 fps (`sample_interval =
max(1, int(fps/2))`), HSV H-S histogram [50,60] bins, `cv2.compareHist(..., HISTCMP_CORREL)`;
a frame-to-frame correlation `< cut\_threshold` (? hard-coded `0.6`, NOT config) counts as a cut.
? HARD CAP: stops after 100 sampled frames (`sampled\_count > 100`) ? only first ~50s analyzed on
long videos. \$config `config["validation"]["max\_scene\_cuts\_per\_minute"]` (default 0.5) is the
fail threshold on computed `cuts\_per\_minute`.
`details={cuts\_detected,sampled\_frames,cuts\_per\_minute}`. ? `cap.set(CAP_PROP_POS_FRAMES,
frame_idx)` per-frame seeking is slow/unreliable on some codecs.

- `BlurValidator` @backend/pipeline/validators/blur.py::BlurValidator ? name `"blur\_check"`.
OpenCV; samples 15 evenly-spaced frames (`step = max(1, total\_frames//15)`), computes Laplacian
variance (`cv2.Laplacian(gray, CV\_64F).var()`) as focus metric, averages. \$config
`config["validation"]["min\_blur\_threshold"]` (default 100.0); fails if `avg_variance <
threshold`. `details={avg_laplacian_variance,min_?,max_?,sampled_frames_count}`. ? Laplacian-
variance threshold is scene/resolution dependent ? 100.0 is a magic absolute value not
normalized to resolution.

- `RelevanceValidator` @backend/pipeline/validators/relevance.py::RelevanceValidator ? name
`"relevance\_check"`. OpenCV; samples 10 frames, downscales to 320x240, HSV `cv2.inRange` court-
green mask, computes per-frame green-pixel ratio, averages. ? INTEGRATION POINT: imports `from
backend.sports import sport_registry` (lazy, inside `validate`), resolves
`config.get("sport","badminton")` ? `sport\_profile.get\_relevance\_config()`; on `KeyError` falls
back to `{}`. \$config precedence (3-tier): `config["validation"]["relevance"][key]` ?
`sport\_rel\_cfg[key]` ? hard-coded default. Keys: `court\_green\_lower` (default `[35,40,40]`),
`court\_green\_upper` (default `[85,255,255]`), `court\_pixels\_min\_ratio` (default 0.05).
`details={avg\_green\_court\_ratio,max\_?,min\_?,green\_threshold}`. ? "relevance" is purely color-
heuristic (green pixel ratio) despite description mentioning "geometric features" ? no geometry
is actually computed. ? assumes green courts; blue/wood badminton courts would fail. ? cross-

subsystem coupling to `backend.sports.sport\_registry`.

Cross-cutting notes

- \$Shared `context` dict is the only inter-validator channel; only `FormatValidator` writes it (`ffprobe\_meta`), only `ResolutionFPSValidator` reads it. PTS/SceneCut/Blur/Relevance each re-open the file independently (ffprobe or OpenCV) ? multiple decode passes per video.
- ? Three external dependencies: `ffprobe` CLI (format, pts_continuity) and `cv2`/`numpy` (scene_cut, blur, relevance). No availability guard ? missing binary surfaces as a per-validator failed result via the registry try/except.
- ? Config schema is implicit/duck-typed: all thresholds live under `config["validation"]` (and `["validation"]["relevance"]`) with inline defaults scattered across files ? no central schema. Two thresholds (PTS 10.0s gap, scene-cut 0.6 correlation) are NOT config-overridable.

Note: I did not find a `tests/` directory among the provided paths (none was listed), so no test-to-subsystem mapping is included. The relevant files are all under
`C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/`.

chunk: Segmenters

Here is the dense CODE_MAP chunk for the Segmenters subsystem.

Segmenters subsystem

Two-stage rally-segmentation plugins. All concrete segmenters subclass `BasePlaySegmenter`, self-register into a module-level `segmenter\_registry` at import time, and conform to one `process\_video(...)` -> List[Dict] contract. The four "real" segmenters share a Phase 2 (local candidate generation) ? Phase 3 (AI provider semantic confirmation) ? Phase 4 (DB population) shape; only Phase 2 differs (person-motion vs shuttle-trajectory).

`@backend/pipeline/segmenters/\_\_\_init\_\_\_py` ? package surface / eager registration

- ? Importing the package eagerly imports all 5 concrete segmenters, which is what triggers their bottom-of-file `segmenter\_registry.register(...)` calls. **Side-effect import is the registration mechanism** ? a segmenter not imported here is invisible to the registry.
- `@backend/pipeline/segmenters/\_\_\_init\_\_\_py::\_\_all\_\_` ? re-exports `segmenter\_registry`, `BasePlaySegmenter`, `MotionPlaySegmenter`, `GeminiPlaySegmenter`, `HybridYoloSegmenter`, `TrackNetHybridSegmenter`, `WasbHybridSegmenter`.

`@backend/pipeline/segmenters/base.py` ? ABC + registry (extension point)

- `@backend/pipeline/segmenters/base.py::BasePlaySegmenter` ? ABC. `\_\_init\_\_(self, db: Database, config: Dict[str, Any])` stores `self.db`, `self.config`.
- \$abstract contract: `name` -> str` (property), `description` -> str` (property), `process\_video(self, video\_id, video\_path, player\_pool: List[str]=None, max\_frames: int=None)` -> List[Dict[str, Any]]. Docstring contract: process video, populate SQLite, **and** return list of segment dicts.
- ? `@backend/pipeline/segmenters/base.py::SegmenterRegistry` ? name?class plugin registry. `register(segmenter\_cls)`, `get(name)` -> Optional[Type], `list\_available()` -> Dict[str, str] (name?description).
- ? `@backend/pipeline/segmenters/base.py::SegmenterRegistry.register` instantiates the class with `segmenter\_cls(None, {})` **just to read its `name`**. This means every segmenter's `name`/`description` properties (and `\_\_init\_\_`) **must be safe to construct with `db=None, config={}`** ? no DB access, no config reads at construction. `HybridYoloSegmenter.\_\_init\_\_` honors this (only sets `self.model=None`). Any `\_\_init\_\_` that touched `db`/`config` would break registration.
- ? `list\_available()` re-instantiates each class with `(None, {})` and swallows all exceptions into "No description available." ? a throwing `description` is silently masked.
- `@backend/pipeline/segmenters/base.py::segmenter\_registry` ? the singleton instance every module registers into.

`@backend/pipeline/segmenters/motion.py` ? classical CV baseline (no AI, no DB candidates)

- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` ? `name="motion"`, frame-difference motion heuristic. Self-contained (no provider/sport/registry deps).
- Pipeline: `cv2.VideoCapture` ? grayscale + `GaussianBlur(21,21)` ? `absdiff`/`threshold(25)`/`dilate` ? motion ratio per frame; analysis sub-sampled to

```
`analysis_fps=5.0` via `cap.grab()` skipping; 5-frame moving-average smooth; threshold-cross
segmentation; merge-then-filter post-processing.
- $config(`config["indexing"]`): `motion_threshold` (0.08), `min_segment_duration` (3.0),
`dead_time_merge_gap` (2.0).
- $DB writes: `db.add_play_segment(video_id, start, end, server, receiver, metadata)` then
per-segment `db.add_event(segment_id, timestamp, event_type, player, details)` for serve /
smash(es) / foul-or-winner.
- $return dict keys: `id, start_time, end_time, duration, server, receiver, metadata`;
`metadata` keys `shot_count, intensity, avg_speed_kmh`.
- ? **Almost everything is fabricated via `random`**: default
`player_pool=["Avi","Suresh","Ramesh","Deepak"]`, server/receiver `random.sample`,
`shot_count`/`intensity`/`avg_speed_kmh`, all serve/smash/foul events and their `speed_kmh`.
This is demo/mock data, NOT real detection output ? contrast with Gemini's "no fabricated data"
stance.
- ? Two distinct progress mechanisms: a `[motion] %`-of-frames stdout line (every ~5%) **and**
a separate `[DEBUG]` print on `max_frames`. Uses bare `print(..., flush=True)`, not `logger`.
```

`@backend/pipeline/segmenters/gemini.py` ? pure-AI segmenter (no local Phase-2 detector)

```
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` ? `name="gemini"`. Sends video
(whole or chunked) straight to an AI provider; no person/shuttle detection.
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter.process_video` ? wiring:
`sport_registry.get(sport_name)` (default `"badminton"`), provider via
`provider_registry.get(provider_name, api_key=, model_name=)`.
- $config: `config["sport"]`; `ai_provider`/`ai_model` read from top-level config OR
`indexing` (defaults `"gemini"`, `"gemini-2.5-flash"`); `indexing.debug_duration`,
`indexing.chunk_duration` (300), `indexing.chunk_overlap` (30), `indexing.min_segment_duration`
(3.0); `stitching.begin_padding`/`end_padding` (0.5 each, used only for the duration *estimate*
log).
- $env contract: requires `f"{provider_name.upper()}_API_KEY"` in `os.environ`; missing key ?
logs and returns `[]` (no raise). Same pattern across all AI segmenters.
- $provider contract: `provider.analyze_video(path, prompt, chunk_duration=, label=) ->
(segments: list, metrics: dict)`. `metrics` keys consumed: `upload_time, process_time, gen_time`
(+ segmenter-injected `file_size_mb, segments_found`).
- $sport contract: `sport_profile.get_prompt(chunk_duration=...)`.
- $AI segment dict keys read: `start_time, end_time, shuttle_exchange_count,
shot_count_confidence`.
- Chunking: only when `video_duration > chunk_duration`; overlapping chunks (step =
`chunk_duration - chunk_overlap`) extracted via `extract_video_chunk`, processed in a
`ThreadPoolExecutor(max_workers=5)` (hard-coded 5), timestamps offset back to full-video coords.
Else single-file path.
- $DB writes: `db.add_play_segment(video_id, start, end, None, None, metadata)`
(server/receiver always `None` ? real detector, no fake players). `metadata` keys: `shot_count,
shot_count_confidence, detector(=model_name)`.
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter.process_video.parse_time`
(nested) ? coerces timestamp to float seconds; handles int/float, plain numeric str, and ? colon
`MM:SS`/`HH:MM:SS` (Gemini sometimes ignores "decimal seconds" instruction ? converts but
warns); unparseable ? `0.0` with warning.
-
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter.process_video.process_single_chunk`
(nested) ? per-chunk extract?analyze?offset?cleanup; returns `(label, segments, metrics)`.
- Validation pass (rich, Gemini-only): reject `start>=end`; reject `start >= video_duration`;
**clamp** `end` to `video_duration`; reject `dur < min_segment_duration`; flag-but-keep `dur >
max_segment_duration` (? hard-coded `max_segment_duration = 90.0`); resolve overlaps by keeping
the **longer** segment.
- $return dict keys: `id, start_time, end_time, duration, metadata`.
- ? Does **not** persist candidate segments (`add_candidate_segment`) ? only the hybrids feed
the fine-tuning table.
```

`@backend/pipeline/segmenters/yolo\_hybrid.py` ? Phase 2 = torchvision person-detect + ByteTrack

```
- Module docstring: ? **name is a lie for back-compat** ? registry key/`detector` tag stay
`"yolo_hybrid"` but no YOLO is used. Ultralytics (AGPL) was swapped for torchvision detectors
(BSD) + supervision ByteTrack (MIT) to keep the hosted service copyleft-free (see
`docs/MONETIZATION_AUDIT.md`, ADR-005).
- ? `@backend/pipeline/segmenters/yolo_hybrid.py::PERSON_CLASS_ID = 1` ? torchvision/COCO
person class is **1** (background is implicit 0); ultralytics used **0**. The classic off-by-one
```

when swapping detectors; explicitly named to prevent regression.

```
- ? `@backend/pipeline/segmenters/yolo_hybrid.py::DETECTOR_FACTORIES` ? config-selectable BSD
detectors: `fasterrcnn_resnet50_fpn`, `fasterrcnn_mobilenet_v3_large_fpn`,
`retinanet_resnet50_fpn`. `@...DEFAULT_DETECTOR` = "fasterrcnn_resnet50_fpn".
- `@backend/pipeline/segmenters/yolo_hybrid.py::fmt_ts` ? `HH:MM:SS` formatter for cross-
verifying timestamps (duplicated verbatim in tracknet_hybrid and wasb_hybrid).
- `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` ? `name="yolo_hybrid"`.
`__init__` sets `self.model=None`, `self.device=None`.
- `@...HybridYoloSegmenter._lazy_load_model` ? ? imports `torchvision.models.detection`
**lazily** so the module imports (and tests run) without torchvision; tests pre-set `self.model`
to skip weight loading. Resolves device from `indexing.use_gpu` (default True) +
`torch.cuda.is_available()`; unknown `indexing.detector_model` ? warn + fall back to default;
`weights="DEFAULT"`.
- `@...HybridYoloSegmenter._make_tracker(fps)` ? fresh `supervision.ByteTrack` per chunk (?
track IDs are **chunk-local** only); lazy `import supervision`.
- `@...HybridYoloSegmenter._detect_and_track(frame, tracker, score_thresh)` ? BGR?RGB?CHW
float[0,1] tensor; `model([tensor])`; mask `labels==_PERSON_CLASS_ID & scores>=score_thresh`; ?
ByteTrack requires `confidence` populated on `sv.Detections`; returns `List[(track_id,
(x1,y1,x2,y2))`, skipping `tracker_id is None`.
- `@...HybridYoloSegmenter._extract_action_windows_from_chunk` ? $returns window dicts
`{start, end, active_frame_count, peak_velocity, mean_velocity, track_id_count, chunk_index}`.
Per-track velocity via `get_velocity_normalised(prev_bbox, bbox, effective_fps, frame_width)`
(from `backend.utils.geometry`); frame counted active if any track exceeds `velocity_thresh`;
stale-ID pruning each frame (? prevents false spikes on ID reassignment); groups frames with gap
? `merge_gap*fps`; filters windows `< indexing.min_action_window_duration` (2.0).
- ? `frame_skip` (default 3): processes every Nth frame; `effective_fps = fps/frame_skip`.
`tracker_config`/`tracker_config="bytetrack.yaml"` param is **dead/back-compat only** (ByteTrack
ignores it).
- `@...HybridYoloSegmenter.process_video` ? full 4-phase orchestration.
- $config (`indexing`): `chunk_duration`(300), `chunk_overlap`(30),
`yolo_velocity_threshold`(0.15, fraction of frame-width/sec), `dead_time_merge_gap`(2.0),
`yolo_frame_skip`(3), `yolo_tracker`("bytetrack.yaml", unused),
`ai_handoff_padding`|`gemini_handoff_padding`(2.0), `ai_max_workers`|`gemini_max_workers`(5),
`log_yolo_candidates`(True), `store_yolo_candidates`(True), `detector_score_threshold`(0.5),
`yolo_prefetch_workers`(2). ? `gemini_*` keys are deprecated aliases still honored.
- Phase 2 has two paths: **pipelined** (`num_chunks>1` and `prefetch_workers>0`) ? ffmpeg
extraction runs in a prefetch `ThreadPoolExecutor` overlapping sequential GPU inference (? GPU
stays sequential, single GPU); else **sequential**.
- Cross-chunk dedup: `@...process_video._merge_stats` (nested) merges overlapping windows
(`current.start <= last.end`); frame-weighted `mean_velocity`, `max` of end/peak/track_count.
- $DB candidate write: `db.add_candidate_segment(video_id, detector="yolo_hybrid",
start_time, end_time, chunk_index, confidence, stats, detection_params)`; `confidence = min(1.0,
peak_velocity)` (? coarse proxy). `stats` keys: `peak_velocity, mean_velocity,
active_frame_count, track_id_count`. `detection_params` snapshot persisted for fine-tuning
reproducibility.
- Phase 3: `@...process_video.process_candidate_window` (nested) ? extract padded handoff
clip with `reencode=True` (? avoids keyframe-boundary drift on short clips),
`provider.analyze_video`, offset timestamps, stamp `segment["_candidate_id"]`. Runs in
`ThreadPoolExecutor(max_workers=ai_max_workers)`; ? iterates futures in **submission order** to
preserve temporal ordering.
- Phase 4 $DB: `db.add_play_segment(video_id, start, end, None, None, metadata)` +
`db.link_candidate_to_segment(candidate_id, segment_id)`. `metadata.detector = f"yolo-
hybrid-{model_name}"`. ? one candidate ? many segments is possible (one-to-many link).
- $return dict keys: `id, start_time, end_time, duration, metadata`.
```

`@backend/pipeline/segmenters/tracknet\_hybrid.py` ? Phase 2 = TrackNetV4 shuttle trajectory (WSL2)

```
- `@backend/pipeline/segmenters/tracknet_hybrid.py::fmt_ts` ? duplicate of the yolo_hybrid
helper.
- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` ?
`name="tracknet_hybrid"`. Phase 3/4 identical to yolo_hybrid; only Phase 2 swaps person-motion
for shuttle trajectory. Detector runs in WSL2 (ADR-001).
- imports `@backend/pipeline/detectors/tracknet_runner.py::TrackNetRunner` +
`::TrackNetConfig`.
- `@...TrackNetHybridSegmenter._probe_fps_width(video_path)` (staticmethod) ? returns `(fps,
frame_width)` with fallbacks `30.0`/`1280.0`.
```

- `@...TrackNetHybridSegmenter.process_video` ? $config:`
``TrackNetConfig.from_indexing_cfg(idx_cfg)`; nested `indexing.tracknet.velocity_threshold` (?
0.02, normalised shuttle displacement/sec ? much smaller than yolo's 0.15 person threshold);
`dead_time_merge_gap`(2.0), `min_action_window_duration`(2.0), `ai_handoff_padding`(2.0),
`ai_max_workers`(5). ? reuses **yolo-named** flags `log_yolo_candidates`/`store_yolo_candidates`
even though no YOLO involved.`
- ? `**No pre-chunking in Phase 2**`: TrackNet processes the whole video in one pass (carries
its own frame buffering), unlike yolo_hybrid.
- Phase 1 ``runner.healthcheck() -> (ok, msg)` fail-fast before any work;`
``runner.run_predict(video_path, out_dir) -> csv_path`; `runner.parse_trajectory_csv(csv) ->`
`points` (each `point.visible`); `runner.trajectory_to_action_windows(points, fps=, frame_width=,
chunk_start=0.0, velocity_thresh=, merge_gap=, min_window_duration=, chunk_index=0)` ? $same
window dict contract as yolo (`track_id_count` present though shuttle has no tracks).`
- \$DB candidate: ``add_candidate_segment(..., detector="tracknet_hybrid", ...)`;`
``detection_params` keys: `detector, velocity_thresh, merge_gap, min_action_window_duration,`
`weights_path, queue_length`.`
- Phase 4 ``metadata.detector = f"tracknet-hybrid-{model_name}"`.`

`@backend/pipeline/segmenters/wasb_hybrid.py` ? Phase 2 = WASB shuttle trajectory (WSL2, live default)`

- Module docstring: ? WASB (nttcom/WASB-SBDT, `**MIT**`) ships `**pretrained badminton weights and`
runs today ? the live shuttle substrate; the per-video `*calibration*` goal mainly uses Phase 2,
tuning ``velocity_thresh`/`merge_gap`/`min_window_duration`` against golden rally labels.
- `@backend/pipeline/segmenters/wasb_hybrid.py::fmt_ts` ? duplicate helper.`
- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` ? `name="wasb_hybrid"`.`
Near-verbatim twin of TrackNetHybridSegmenter.
- imports `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` + `::WasbConfig`.`
- `@...WasbHybridSegmenter._probe_fps_width` (staticmethod) ? same as tracknet.`
- `@...WasbHybridSegmenter.process_video` ? $config: `WasbConfig.from_indexing_cfg(idx_cfg)`;`
nested ``indexing.wasb.velocity_threshold`(0.02); same`
``dead_time_merge_gap`/`min_action_window_duration`/`ai_handoff_padding`/`ai_max_workers`; ? same`
reused yolo-named ``log_yolo_candidates`/`store_yolo_candidates`` flags. Same single-pass (no
chunking), healthcheck-first, runner API (``healthcheck`, `run_predict`, `parse_trajectory_csv`,
`trajectory_to_action_windows`).`
- \$DB candidate: ``detector="wasb_hybrid"`; `detection_params` keys: `detector,`
`velocity_thresh, merge_gap, min_action_window_duration, weights_path` (? no `queue_length`,
unlike tracknet).`
- ? `**Divergent shot_count logic**`: wasb calls ``int(shot_count)`` directly inside the try (no
``is None`` short-circuit) ? ``int(None)`` raises ``TypeError`` and is caught ? falls back to ``max(3,`
`int(dur/0.8))``. tracknet/yolo/gemini check ``is None`` first. Functionally equivalent for ``None``,
but a subtle copy-paste drift.
- Phase 4 ``metadata.detector = f"wasb-hybrid-{model_name}"`.`

Cross-cutting contracts / footguns (apply to all 4 AI segmenters)

- `$**Window dict**` (Phase 2 output, common): `{start, end, active_frame_count, peak_velocity,`
`mean_velocity, track_id_count, chunk_index}`.`
- `$**AI segment dict**` (provider output, keys read): ``start_time, end_time,`
`shuttle_exchange_count?, shot_count_confidence?; segmenter injects `_candidate_id`.`
- `$**Return value**` (all segmenters): ``List[{id, start_time, end_time, duration, metadata}]` (+
`server, receiver` for motion only).`
- `$**DB API surface**` used: ``add_play_segment(video_id, start, end, server, receiver, metadata)`
`-> segment_id`, `add_event(...)` (motion only), `add_candidate_segment(...)` -> candidate_id`
(hybrids only), `link_candidate_to_segment(candidate_id, segment_id)` (hybrids only).`
- ? All AI segmenters `**return `[]`` rather than raising** on: unknown sport, missing API key,
unknown provider, invalid/zero ``video_duration``, failed detector healthcheck/trajectory. Silent-
degradation is the house style ? callers must distinguish "no rallies" from "config error" out-
of-band (logs only).
- ? ``_fmt_ts`` is duplicated in 3 files (yolo/tracknet/wasb) ? a refactor magnet.
- ? tracknet/wasb reuse ``log_yolo_candidates`/`store_yolo_candidates`` config keys; renaming them
would silently break those two segmenters' candidate persistence.

Note: the task listed only segmenter source files; ``tests/`` was not in scope and no test
directory was among the provided paths, so no test-to-subsystem mapping is included. The
referenced runners (``backend/pipeline/detectors/tracknet_runner.py`,
`backend/pipeline/detectors/wasb_runner.py`)` own the ``TrackNetRunner/TrackNetConfig`` and
``WasbRunner/WasbConfig`` contracts referenced above but are outside this chunk.

chunk: Detectors / stitcher / audio (WSL-quarantined)

Here is the dense CODE_MAP chunk for the subsystem.

Detectors / stitcher / audio (WSL-quarantined)

Self-hosted shuttle-trajectory detectors run inside WSL2 (ADR-001); Windows side is a subprocess adapter only ? no TensorFlow/torch imported on Windows. All detectors emit the same `Frame,Visibility,X,Y` CSV (\$ trajectory CSV) ? shared windowing ? optional rally gate ? AI handoff ? DB. Stitcher and audio (E1 probe, not adopted) are separate.

`@backend/pipeline/detectors/tracknet\_runner.py` ? TrackNet WSL adapter + shared trajectory?windows logic

- `@backend/pipeline/detectors/tracknet\_runner.py::TrackNetConfig` ? `@dataclass` of resolved WSL settings (`distro`, `repo\_dir`, `conda\_sh`, `conda\_env`, `weights\_path`, `queue\_length`, `stage\_in\_wsl`, `wsl\_stage\_dir`, `timeout\_sec`).
- ? `TrackNetConfig.from\_indexing\_cfg(idx\_cfg)` ? builds from `config.json` ? indexing.tracknet` (note key-name drift: `wsl_distro` in JSON ? `distro` field).
- ? `weights\_path` defaults to `""` and is REQUIRED; `run\_predict` aborts (returns None) if unset.
- ? `timeout\_sec=0` means *no timeout* (`self.cfg.timeout\_sec` or None).
- `@backend/pipeline/detectors/tracknet\_runner.py::to\_wsl\_mnt\_path(win\_path)` ? Windows `C:\...` ? `/mnt/c/...`; POSIX/~ paths pass through unchanged. Shared helper, also imported by `wasb\_runner`.
- ? Only translates drive-letter paths; UNC / network paths are returned with backslashes?slashes but no `/mnt` prefix.
- `@backend/pipeline/detectors/tracknet\_runner.py::TrajectoryPoint` ? \$ per-frame point `@dataclass`: `frame:int`, `visible:bool`, `x:float`, `y:float`. The in-memory shape of the trajectory CSV; consumed by `rally\_gate`.
- `@backend/pipeline/detectors/tracknet\_runner.py::TrackNetRunner` ? runs `src/predict.py` in WSL, converts CSV ? action windows.
- `TrackNetRunner.\_wsl\_bash(inner\_cmd)` ? `wsl -d <distro> bash -c "source <conda\_sh> && <cmd>"; `subprocess.run(capture_output, text)`.
- `TrackNetRunner.healthcheck()` ? `(ok, msg)` ? activates env, imports TF, prints GPU count. Catches `FileNotFoundError` (no `wsl`) + `TimeoutExpired`.
- `TrackNetRunner.run\_predict(video\_win\_path, output\_win\_dir)` ? Windows CSV path | None. Stages video (if `stage\_in\_wsl`), runs `predict.py --video\_path --model\_weights --output\_dir --queue\_length`.
- ? Only accepts `.mp4`/`.avi`; predict.py hard-names output `\_predict.csv` ? runner reconstructs that exact name to read it back via the `/mnt` view (no copy-back).
- `TrackNetRunner.\_stage\_video(video\_win\_path, base)` ? `cp` from `/mnt` into `wsl\_stage\_dir`; success sentinel is literal "OK" in stdout.
- `TrackNetRunner.parse\_trajectory\_csv(csv\_win\_path)` `@staticmethod` ? `List[TrajectoryPoint]`. \$ parses `Frame,Visibility,X,Y`; `Visibility==1` ? visible; silently skips rows on `KeyError`/`ValueError`; sorts by frame. **Shared** (WASB rebinds this).
- `TrackNetRunner.trajectory\_to\_action\_windows(points, fps, frame\_width, chunk\_start, velocity\_thresh=0.02, merge\_gap=2.0, min\_window\_duration=2.0, chunk\_index)` `@staticmethod` ? `List[Dict]`. \$ window dict: `start`, `end`, `active\_frame\_count`, `peak\_velocity`, `mean\_velocity`, `track\_id\_count` (=1), `chunk\_index`. Frame "active" when visible AND normalised inter-frame velocity > thresh; invisible frame breaks the trajectory (`prev=None`); active frames within `merge\_gap` grouped; short windows dropped. Mirrors HybridYoloSegmenter window shape. **Shared** (WASB rebinds this).
- ? `fps<=0`?30.0, `frame\_width<=0`?1280.0 silent fallbacks.

`@backend/pipeline/detectors/wasb\_runner.py` ? WASB WSL adapter (reuses TrackNet windowing)

- `@backend/pipeline/detectors/wasb\_runner.py::\_INFER\_WIN` ? module-level const: absolute Windows path to sibling `wasb\_infer.py` (staged into WSL each run).
- `@backend/pipeline/detectors/wasb\_runner.py::WasbConfig` ? `@dataclass`: `distro`, `repo\_dir` (~/models/WASB-SBDT), `conda\_sh`, `conda\_env` (`wasb`), `weights\_path` (defaults to `wasb\_badminton\_best.pth.tar`), `sport` (badminton), `wsl\_stage\_dir`, `timeout\_sec`.
- ? `WasbConfig.from\_indexing\_cfg(idx\_cfg)` ? from `config.json` ? indexing.wasb`.
- ? Unlike TrackNet, `weights\_path` has a real default ? no required-field guard in `run\_predict`.
- `@backend/pipeline/detectors/wasb\_runner.py::WasbRunner` ? stage ? infer ? CSV.

```
- `WasbRunner._wsl_bash` / `WasbRunner.healthcheck` / `WasbRunner._stage_video` ? mirror
TrackNet's (healthcheck imports torch + `torch.cuda.is_available()`).
- `WasbRunner._stage_infer_script()` ? `cp` `__INFER_WIN` ? `/src/wasb_infer.py` so
it can import WASB modules. ? Overwrites on every run; "OK" sentinel.
- `WasbRunner.run_predict(video_win_path, output_win_dir)` ? CSV path | None. Stages infer
script + video, extracts to `

```

`@backend/pipeline/detectors/wasb\_infer.py` ? WASB inference core (runs INSIDE WSL `wasb` env; resumable)

```
- ? Imports `detectors`/`trackers`/`dataloaders`/`utils` from the WASB-SBDT repo ? only
importable from `:08d>.png`. Idempotent: skips if existing PNG count ? container's reported frame
count. ? cv2 PNG extraction is slow ? prefer host ffmpeg + `--frames_dir` for big clips.
- `@backend/pipeline/detectors/wasb_infer.py::main()` ? argparse CLI; `--video` (internal
extract) XOR `--frames_dir`; flags `--weights` (req), `--out` (req), `--sport`, `--limit`, `--batch-
size`, `--num-workers`, `--log-every-batches`, `--cache-dir`, `--flush-every`, `--fresh`.
```

`@backend/pipeline/detectors/rally_gate.py` ? Gate A: net-crossing post-filter (pure, no I/O, no model)

- Suppresses between-point false fires: real rally = shuttle crosses net MULTIPLE times; single service feed crosses ~once. Operates on the trajectory we already have (`TrajectoryPoint`'s). ? Gate A in the over-fire spectrum; Gate B (player-stance state machine) is future/unbuilt.
- `@backend/pipeline/detectors/rally_gate.py::Interval` ? type alias `Tuple[float, float]` (start, end seconds).
- `@backend/pipeline/detectors/rally_gate.py::GatedWindow` ? `@dataclass`:
`start`, `end`, `crossings`, `visible_points`, `kept`.
- `@backend/pipeline/detectors/rally_gate.py::spread(vals)` ? robust p95?p5.
- `@backend/pipeline/detectors/rally_gate.py::median(vals)` ? median (used as net estimate).
- `@backend/pipeline/detectors/rally_gate.py::estimate_play_axis(points)` ? (axis 'x'|'y', net_value, span). Play axis = larger-spread image axis; net = median coord on it (shuttle clusters at ends, median ? net). ? Ties go to 'x' ('sx >= sy').
- `@backend/pipeline/detectors/rally_gate.py::count_net_crossings(window_points, axis, net, deadband)` ? sign changes of (coord - net) across visible points; deadband zone near net ignored (jitter suppression).
- `@backend/pipeline/detectors/rally_gate.py::points_in(points, fps, start, end)` ? ? filters by `start*fps <= p.frame <= end*fps` (window times are seconds, trajectory is frame-indexed ? fps coupling).
- `@backend/pipeline/detectors/rally_gate.py::gate_windows(points, windows, fps, min_crossings=2, deadband_frac=0.06)` ? `List[GatedWindow]`. Axis/net estimated ONCE over whole trajectory (stable); deadband = `deadband_frac * span`; `kept = crossings >= min_crossings`.
- `@backend/pipeline/detectors/rally_gate.py::apply_gate(...)` ? `List[Interval]` ? convenience: only kept windows. ? Window contract here is (start, end) tuples, NOT the rich window dict from `trajectory_to_action_windows` ? caller must adapt between the two shapes.

`@backend/pipeline/stitcher/compiler.py` ? FFmpeg highlight compiler (Windows-side, no WSL/model)

- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? trims selected segments and concatenates into one reel.
- `VideoCompiler.__init__(output_dir, end_padding=1.0, begin_padding=0.5)` ? makes `output_dir`; padding applied around every segment.
- `VideoCompiler.parse_time(val)` `@staticmethod` ? float seconds. \$ accepts float/int, numeric strings, and colon forms (`MM:SS`/`HH:MM:SS`). Mirrors `gemini.py::parse_time` so stitcher consumes whatever the indexer emitted. ? Unparseable ? warns + returns `0.0` (silent default, not an error).
- `VideoCompiler.get_video_duration(video_path)` ? `ffprobe format=duration`; returns `0.0` (unknown) on any failure ? boundary validation then skipped.
- `VideoCompiler.compile_highlights(raw_video_path, segments, output_filename)` ? output path. \$ `segments: List[Tuple[float, float]]`. Per-segment: normalise times, clamp start<0?0, skip `start>=end`, skip start-beyond-duration, clamp end/padded_end to duration. Trims each to a temp `clip_{idx}.mp4` (re-encode: `libx264 -preset superfast -crf 22`, `aac 128k`), then concat-demuxer copy (`-c copy`) into `output_filename`.
- ? Two-stage codec contract: clips are re-encoded with *identical* settings precisely so the final concat can be stream-copy (`-c copy`) with no re-encode ? change one preset and concat may break.
- ? Raises `ValueError` (empty `segments`), `FileNotFoundError` (missing source), `RuntimeError` (all segments skipped, or concat produced empty/missing output). Individual bad segments are skipped + summarised, not fatal.
- ? Uses a `TemporaryDirectory(dir=output_dir)` for clips + `clips.txt` concat list (single-quotes path-escaped).

`@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector (E1 probe; NOT adopted for fusion)

- Stages 0?1 of `docs/AUDIO_RALLY_DETECTION_PLAN.md`. Pure numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. ? Separate signal ? WASB is never modified by this. Per audio-e1-findings: recall 100% but discrimination only ~2.2x, band-pass null ? NOT adopted for fusion (ADR-007).
- `@backend/pipeline/audio/hit_detector.py::HitEvent` ? \$ `@dataclass`: `t:float` (seconds), `strength:float` (onset-envelope peak, arbitrary units).
- `@backend/pipeline/audio/hit_detector.py::extract_audio(video_path, out_wav=None, sr=16000)` ? ffmpeg ? mono 16-bit PCM WAV (`-vn -ac 1 -ar sr -c:a pcm_s16le`); temp file if `out_wav` unset; audio-only decode (fast on multi-GB video).
- `@backend/pipeline/audio/hit_detector.py::load_wav(path)` ? `(float32 [-1,1], sr)`. Handles sample widths 2/4/1 bytes; downmixes multichannel by mean. ? Raises `ValueError` on unsupported sample width.
- `@backend/pipeline/audio/hit_detector.py::preemphasis(x, coef=0.97)` ? boosts broadband HF

transient of racket-shuttle contact.

```
- `@backend/pipeline/audio/hit_detector.py::onset_envelope(samples, sr, frame=1024, hop=256,
band=None)` ? `(env, hop_sec)` Pure-numpy STFT (Hanning, `np.fft.rfft`) ? positive spectral
flux. $ `band=(lo_hz,hi_hz)` band-limits flux (e.g. 379 kHz) to shuttle-click range. ? Needs
`sr` high enough for `hi_hz` (Nyquist=sr/2); returns empty env if `len(x)<frame`.
- `@backend/pipeline/audio/hit_detector.py::_moving_stats(env, win)` ? `(mean, std)` Sliding-
window via cumsum, reflect-padded; ? `win` forced odd.
- `@backend/pipeline/audio/hit_detector.py::detect_hits(samples, sr, k=2.5, win_frames=41,
min_gap_sec=0.08, band=None)` ? `List[HitEvent]`. Local maxima of onset env exceeding adaptive
threshold `mean + k*std`; greedily keep strongest within `min_gap_sec`. ? `k` is the sensitivity
knob (lower = more hits); `band` defaults None (full-spectrum ? band-pass found null in E1).
- `@backend/pipeline/audio/hit_detector.py::detect_hits_in_video(video_path, sr=16000, **kw)` ?
`List[HitEvent]` ? extract + detect + cleanup temp wav (finally unlink, swallows `OSError`).
```

Notes / cross-cutting

```
- ? **Detector interchangeability extension point:** the WSL detector "plugin" contract is the $
`Frame,Visibility,X,Y` CSV + the two shared statics (`parse_trajectory_csv`,
`trajectory_to_action_windows`). A new detector adds an adapter like `wasb_runner.py` and
rebinds those two ? no downstream change. There is no central registry/dispatch in these files;
selection is presumably done in the indexer (not in this subsystem).
- $ **Window-shape mismatch** between `trajectory_to_action_windows` (rich dict) and
`rally_gate` (`(start,end)` tuple) ? a real adapter seam, flagged above.
- ? **WSL quarantine:** `tracknet_runner.py` and `wasb_runner.py` import no TF/torch and run on
Windows; `wasb_infer.py` imports torch + WASB repo modules and runs ONLY in WSL. Don't import
`wasb_infer` from Windows code.
- **Tests:** no test directory was in the provided file list, so none are enumerated here. Files
outside this subsystem referenced but not read: the indexer/segmenter that selects a detector
and the AI-handoff (`gemini.py`, referenced by `compiler._parse_time`).
```

(Note: I did not find a `tests/` path among the inputs you gave ? the prompt mentioned mapping a
`tests/` directory, but none was in the file list, so there is nothing to enumerate. If you want
test coverage mapped, point me at the test directory.)

chunk: Infrastructure & reporting (observability)

@backend/infrastructure/database.py

```
- `Database` class ? single SQLite gateway; opens a fresh connection per call.
```

@backend/infrastructure/database.py::Database

```
- `Database.__init__(db_path)` ? stores `db_path`, calls `_init_db()` on construction.
```

@backend/infrastructure/database.py::Database.__init__

```
- `Database._get_connection()` ? `sqlite3.connect`, sets `row_factory = sqlite3.Row`, runs
`PRAGMA foreign_keys = ON`. ? FK enforcement is per-connection in SQLite and OFF by default ?
every connection MUST re-run this PRAGMA or `ON DELETE CASCADE`/`SET NULL` silently won't fire.
```

@backend/infrastructure/database.py::Database._get_connection

```
- `Database._init_db()` ? creates 3 base tables + runs versioned migration block. ? Migration
contract: `PRAGMA user_version` is the schema version; bump it for every change and add an
ordered `if version < N:` block; tests assert the expected version so accidental drift fails CI.
Only `version < 1` exists today (creates `candidate_segments`).
```

@backend/infrastructure/database.py::Database._init_db

```
- $ `videos` table ? `id` TEXT PK (file MD5), `filepath`, `processed_at`, `status`,
`validation_results` (JSON text). @backend/infrastructure/database.py (DDL in `_init_db`)
```

```
- $ `play_segments` table ? confirmed segments; `id` INTEGER PK AUTOINCREMENT, `video_id`
FK?videos `ON DELETE CASCADE`, `start_time`/`end_time`/`duration` REAL, `server`, `receiver`,
`metadata` (JSON text, extensibility point). @backend/infrastructure/database.py (DDL in
`_init_db`)
```

```
- $ `events` table ? `id` INTEGER PK, `segment_id` FK?play_segments `ON DELETE CASCADE`,
`timestamp`, `type`, `player`, `details` (JSON text). @backend/infrastructure/database.py (DDL
in `_init_db`)
```

```
- $ `candidate_segments` table (schema v1) ? raw pre-confirmation detections, detector-agnostic.
Cols: `video_id` FK?videos CASCADE, `detector` TEXT NOT NULL, `chunk_index`,
`start_time`/`end_time`/`duration`, `confidence`, `stats` (JSON), `detection_params` (JSON),
`confirmed_segment_id` FK?play_segments `ON DELETE SET NULL`, `human_label`, `notes`,
`created_at`. ? Plugin extension point: a new segmenter reuses this table by setting its own
```

```

`detector`/`stats`/`detection_params` ? common fields are columns, detector-specific metrics go
in `stats` JSON. Index `idx_candidates_video_detector` on `(video_id, detector)`.
@backend/infrastructure/database.py (DDL in `_init_db`)
- `Database.add_video(video_id, filepath, status, validation_results)` ? `INSERT OR REPLACE`
(re-ingest path); JSON-encodes `validation_results`; returns bool. ? `INSERT OR REPLACE` on the
videos PK triggers the CASCADE delete of prior `play_segments`/`events` ? this is the re-ingest-
replaced-prior behavior the report flags.
@backend/infrastructure/database.py::Database.add_video
- `Database.add_play_segment(video_id, start_time, end_time, server=, receiver=, metadata=)` ?
computes `duration = end_time - start_time`; JSON-encodes `metadata`; returns new rowid or None.
@backend/infrastructure/database.py::Database.add_play_segment
- `Database.add_event(segment_id, timestamp, event_type, player=, details=)` ? note param is
`event_type` but the column is `type`; JSON-encodes `details`; returns bool.
@backend/infrastructure/database.py::Database.add_event
- `Database.add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=,
confidence=, stats=, detection_params=)` ? persists one raw detection;
`stats`/`detection_params` JSON-encoded; returns candidate id or None.
@backend/infrastructure/database.py::Database.add_candidate_segment
- `Database.link_candidate_to_segment(candidate_id, segment_id)` ? sets `confirmed_segment_id`
on a candidate (confirmation linkage for the fine-tuning workflow); returns bool.
@backend/infrastructure/database.py::Database.link_candidate_to_segment
- `Database.query_candidate_segments(video_id, detector=None, only_unlabeled=False)` ? dynamic
WHERE; `only_unlabeled` filters `human_label IS NULL`; deserializes `stats`/`detection_params`
from JSON; ordered by `start_time`.
@backend/infrastructure/database.py::Database.query_candidate_segments
- `Database.get_video(video_id)` ? single row; deserializes `validation_results` (default
`""`); returns dict or None. @backend/infrastructure/database.py::Database.get_video
- `Database.query_play_segments(video_id, filters)` ? S filter contract: `duration_min` (float),
`server` (str), `receiver` (str), `event_type` (str, matched via `EXISTS` subquery on
`events.type`). Hydrates each segment's `metadata` JSON and nested `events` (with `details`
JSON). ? All filter keys use truthiness (`filters.get(...)`) so `duration_min=0`, empty-string
server/receiver are treated as "no filter".
@backend/infrastructure/database.py::Database.query_play_segments
- `Database.clear_video_data(video_id)` ? `DELETE FROM videos WHERE id = ?`; relies on CASCADE
to purge segments/events/candidates.
@backend/infrastructure/database.py::Database.clear_video_data

@backend/reporting/_init_.py
- Module docstring contract: `obsreport` is a SOFT dependency ? absent ? no-op;
`emit_indexer_report` is designed for a `finally` block and NEVER raises. ? Emitting a report
must not break a processing run. @backend/reporting/_init_.py
- `OBSREPORT_AVAILABLE` const ? set True only if `import obsreport` AND `from . import harvest`
both succeed; any exception ? False. ? Soft-dependency gate that toggles the whole reporting
subsystem. @backend/reporting/_init_.py::OBSREPORT_AVAILABLE
- `report_enabled(config)` ? reads `config["report"]["enabled"]`, default True.
@backend/reporting/_init_.py::report_enabled
- `emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None,
segmenter_requested=None, segmenter_actual=None, was_reingest=False, segmentation_ran=True)` ?
single entrypoint. Normalizes a typed config (calls `config.model_dump(mode="json")` if
`hasattr(config, "model_dump")`, e.g. `backend.config.AppConfig`) to a plain dict; short-
circuits to None if unavailable/disabled; pulls `play_segments`/`candidates` from `db`;
delegates to `harvest.record_run`; writes via
`rec.write(output_dir=config["report"]["output_dir"])`; returns `res.paths` or None. ? Keyword-
only args; wraps everything in a bare `except Exception: return None` ? failures are swallowed
silently by design. @backend/reporting/_init_.py::emit_indexer_report

@backend/reporting/harvest.py
- Module docstring contract: pure-ish harvest ? records raw FACTS into an
`obsreport.RunRecorder`; the footgun RULES (signals?flags) live declaratively in spec.yaml, not
here. Imported only after `_init_` confirms obsreport is installed.
@backend/reporting/harvest.py
- ? `_AI_SEGMENTERS` const ? `{ "yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini" }`; the
set that classifies a run as AI-based (drives `_ai_stage`). Extension point: add new AI
segmenters here. @backend/reporting/harvest.py::_AI_SEGMENTERS
- `_SPEC_PATH` const ? `spec.yaml` resolved next to this module.

```

```

@backend/reporting/harvest.py::_SPEC_PATH
- `load_spec() -> obsreport.ReportSpec` ? `obsreport.load_spec(_SPEC_PATH)`.
@backend/reporting/harvest.py::load_spec
- `video_meta_from_context(val_context, video_path)` ? derives VideoMeta from the validators'
shared `val_context["ffprobe_meta"]` (streams[0] for w/h/r_frame_rate/codecs, format for
duration/container) + `os.path.getsize` for `size_mb`. $ Sets `fps_source` ? {`unknown`,
`probed`} (later promoted to `fallback`). ? Parses fps from the `num/den` `r_frame_rate`
string; guards `ValueError`/`ZeroDivisionError`.
@backend/reporting/harvest.py::video_meta_from_context
- `_validation_stage(rec, val_results)` ? stage `"validation"`; metrics
`checks_passed`/`checks_total`; appends `FAILED <name>: <msg>` per failure; status
`error`/`ok`/`not_run`. ? Reads results via `getattr` duck-typing (`passed`, `validator_name`,
`message`) ? results need not be a fixed type. @backend/reporting/harvest.py::_validation_stage
- `_segmentation_stage(rec, play_segments, candidates, segmenter_requested, segmenter_actual,
config_default, was_reingest, ran)` ? stage `"segmentation"`; metrics `candidate_windows`,
`segment_count` (audience both), `total_play_s` (unit s); reads `detector` tag from
`play_segments[0].metadata`. $ Sets `details`: `segmenter_requested`, `segmenter_actual`,
`config_default`, `segmenter_explicitly_requested`, `matches_config_default`, `was_reingest`,
`detector`, and `metadata_source`. ? `matches_default` only computed when `ran` AND
`segmenter_actual` AND `config_default` (defaults True otherwise) ? an explicit override or halted
run is deliberately NOT a divergence. ? `metadata_source` marks `motion` segmenter output as
"synthetic/heuristic ? NOT measured". @backend/reporting/harvest.py::_segmentation_stage
- `_ai_stage(rec, config, segmenter_actual, play_segments, ran)` ? stage `"ai_handoff"`;
`ai_based` = `segmenter_actual` in `_AI_SEGMENTERS`; derives `provider` = `config["ai_provider"]` or
"gemini", key env `f"{provider.upper()}_API_KEY"`. $ `details`: `ai_based`, `provider`,
`model`, `api_key_present` (`bool(os.environ.get(key_env))`); metric `confirmed_segments` only
when `ai_based`. ? `api_key_present` is the exact signal `silent_provider_noop` keys on; status
`not_run` (not ran) vs `skipped` (ran but non-AI). @backend/reporting/harvest.py::_ai_stage
- `_highlights(rec, play_segments, video_meta)` ? computes `coverage_pct` =
100*total/duration_s; emits a "No highlights were produced" note when empty; calls
`rec.set_highlights(...)`. @backend/reporting/harvest.py::_highlights
- `record_run(*, video_id, video_path, config, val_results, val_context, play_segments,
candidates, segmenter_requested=None, segmenter_actual=None, was_reingest=False,
segmentation_ran=True, tool_version=None, spec=None, **recorder_kwargs) ->
obsreport.RunRecorder` ? top-level assembler (does NOT write). Builds `RunRecorder("badminton-
highlight-indexer", video_id_kind="file_md5", spec=spec or load_spec(), ...)`; reads
`config_default` = `config["indexing"]["default_segmenter"]`; runs the 4 stage builders +
`_highlights`; calls `rec.finalize()`. ? fps fallback: if `fps_source` == "unknown" and
`segmentation_ran`, promotes to "fallback" and forces `fps` = 30.0 (mirrors runners'
`_probe_fps_width` ? 30.0/1280) so the report reads "30 (fallback)" not "? (fallback)".
@backend/reporting/harvest.py::record_run

```

```

@backend/reporting/spec.yaml
- $ Report layout/spec consumed by `obsreport` engine (engine = config language + renderers;
this file = indexer-specific layout + footgun rules). Top-level: `name: indexer`, `title`.
@backend/reporting/spec.yaml
- $ `customer_verdict` ? messages for `pass` / `pass_with_warnings` / `fail`.
@backend/reporting/spec.yaml::customer_verdict
- $ `sections` ? ordered list of `{id, title, kind, audience}`: `run` (identity/both), `video`
(video_meta/both), `highlights` (highlights/customer), `cust_notes` (flags/customer), `stages`
(stages/technical), `findings` (flags/technical), `verdict` (verdict/both). ? `kind` selects an
obsreport renderer; `audience` ? {both, customer, technical}.
@backend/reporting/spec.yaml::sections
- ? $ `rules` ? declarative footgun engine: each rule is `{code, severity, when[{path, op,
value}], message, faq_ref, customer_message?}`. `when` clauses are AND'd; `path` dotted-
navigates the recorder JSON (e.g. `stages.ai_handoff.details.api_key_present`,
`stages.segmentation.metrics.segment_count`, `video_meta.fps_source`). ? These `path` strings
are a hard coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py
silently breaks the matching rule. Extension point: add new rules here, not in code.
@backend/reporting/spec.yaml::rules
- Rule `silent_provider_noop` (error) ? `api_key_present == false` AND `segment_count == 0` ?
missing-credential, not "no rallies"; faq Q7; has `customer_message`.
@backend/reporting/spec.yaml::rules.silent_provider_noop
- Rule `ai_empty_vs_no_rallies` (warn) ? `ai_based == true` AND `api_key_present == true` AND
`segment_count == 0` ? provider error/quota/WSL healthcheck; faq Q7.

```

```
@backend/reporting/spec.yaml::rules.ai_empty_vs_no_rallies
- Rule `fps_fallback_used` (warn) ? `video_meta.fps_source == fallback` ? timing/motion
thresholds may be miscalibrated; faq Q14. @backend/reporting/spec.yaml::rules.fps_fallback_used
- Rule `synthetic_motion_metadata` (warn) ? `segmentation.details.segmenter_actual == motion` ?
per-segment metadata are placeholder/unmeasured; faq Q10.
@backend/reporting/spec.yaml::rules.synthetic_motion_metadata
- Rule `segmenter_divergence` (info) ? `matches_config_default == false` AND
`segmenter_explicitly_requested == false` ? a fallback default kicked in; defaults known to
diverge across config.json/setup.py/CLI; faq Q7.
@backend/reporting/spec.yaml::rules.segmenter_divergence
- Rule `reingest_replaced_prior` (info) ? `was_reingest == true` ? re-process replaced prior
segments (keyed by file MD5, cascade delete); faq Q11.
@backend/reporting/spec.yaml::rules.reingest_replaced_prior
- Rule `validation_failed` (error) ? `stages.validation.status == error` ? indexing halted; faq
Q13; has `customer_message`. @backend/reporting/spec.yaml::rules.validation_failed
```

Cross-cutting notes:

- ? Coupling chain: harvest.py writes stage/metric/detail names ? spec.yaml
`rules[].when[].path` reads them by dotted string. The pairs that MUST stay in sync:
`ai\_handoff.details.{api\_key\_present,ai\_based}`, `segmentation.metrics.segment\_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
- ? The `db` arg to `emit\_indexer\_report` is the `Database` instance from database.py ?
reporting calls only `query\_play\_segments(video\_id, {})` and
`query\_candidate\_segments(video\_id)` on it (read-only).
- No `tests/` directory was in the supplied path list, so no test-to-subsystem mapping is
included.

chunk: Eval & calibration

I have all the information needed. Here is the dense, anchor-rich CODE_MAP chunk for the Eval & calibration subsystem.

Eval & calibration (`backend/eval/`)

Segmenter-agnostic measurement, threshold calibration, Gemini-distillation, and regression-guard layer. Pure-metrics core (no I/O) feeds CLI drivers that turn cached WASB trajectories + golden/silver labels into tuned configs, exported segments, learned classifiers, and "+X%" headlines. Legend: \$=data contract, ?=registry/extension point, ?=footgun.

Pure metrics core (no I/O, deterministic, unit-tested)

- `@backend/eval/metrics.py` ? interval-matching primitives. `interval\_iou(a,b)` temporal IoU (0.0 if degenerate/non-overlapping); `match\_intervals(preds,gts,iou\_threshold=0.5)` greedy ****one-to-one**** matching by descending IoU;
`score(preds,gts,iou\_threshold=0.5,match\_result=None)` ? P/R/F1 + `mean\_iou` + boundary errors;
`pr\_curve(preds,gts,thresholds=None)` sweeps 0.1..0.9.
 - `\$Interval = Tuple[float,float]` ? `(start,end)` seconds, ****the shared currency of the whole subsystem****.
 - `\$@backend/eval/metrics.py::Match` (`pred\_index,gt\_index,iou`); `\$MatchResult` (`matches`, `unmatched\_pred\_indices`=FPs, `unmatched\_gt\_indices`=FNs); `\$Score` (dataclass: `iou\_threshold,num\_pred,num\_gt,true\_positives,false\_positives,false\_negatives,precision,recall,f1,mean\_iou,mean\_start\_error,mean\_end\_error`; `.as\_dict()`).
 - ? matching is greedy (highest-IoU-first), not optimal/Hungarian ? ties broken by `(pi,gi)` for determinism. ? `score` with empty matches returns 0.0 for `mean\_iou`/boundary errors, not NaN.
- `@backend/eval/annotations.py` ? ground-truth CSV loader. `parse\_timestamp(value)` accepts decimal sec OR `MM:SS`/`HH:MM:SS` (mixed OK), raises `ValueError` on unparseable;
`load\_annotations(csv\_path)` ? sorted intervals; `write\_scaffold(csv\_path)` writes header-only file.
 - Sannotation CSV: `start,end` header optional/auto-skipped; `#` comments + blanks ignored; ? raises (not skips) on malformed row or `start >= end` ? labeling mistakes fail loudly. ? this `start,end` schema differs from the golden `start\_time/end\_time` schema in `calibrate\_wasb.load\_golden\_gts`.

```
- `@backend/eval/calibration.py` ? overfit-guarded improvement measurement (pure, segmenter-agnostic via `predict_fn`). `pct_improvement(before,after)`; `evaluate(preds,gts,iou=0.5)`;
`kfold_indices(n,k)` deterministic round-robin split;
`select_best(predict_fn,configs,gts,metric="f1",iou=0.5)`;
`cross_validate(...,k=5,metric="recall")` within-video k-fold ? `generalization_gap`;
`leave_one_video_out(videos,configs,metric="f1")` ? `mean_held_out`;
`improvement_report(predict_fn,baseline_config,tuned_config,gts)`.
- `$Config = Any` (opaque); `$PredictFn = Callable[[Config], List[Interval]]` ? **the plug-in seam** all drivers conform to. ? any segmenter (WASB/yolo_hybrid/learned) participates by supplying a `predict_fn`.
- `$leave_one_video_out` videos contract: `{"name":str, "predict_fn":PredictFn, "gts":[Interval]}`.
- ? `cross_validate` default metric is **recall** on purpose ? predictions are whole-video, so precision/F1 are ill-defined per-fold (other rallies' windows count as FPs). ?
`generalization_gap > ~0.1` = likely overfit. ? `pct_improvement` returns `inf` when `before<=0` and `after>0`.
```

Calibration / export CLI drivers (I/O + side effects; thin glue over the core)

```
- `@backend/eval/calibrate_wasb.py` ? per-video WASB threshold calibration driver.
`run(trajecory_csv, labels_csv, fps, frame_width, iou=0.5)`;
`make_predict_fn(trajecory_csv, fps, frame_width)` parses trajectory ONCE then closes over
`TrackNetRunner.trajecory_to_action_windows`; `load_golden_gts(csv_path)` ($golden CSV:
`start_time`/`end_time` cols, case/space-normalized, silently skips unparseable rows);
`default_grid()` 5x3x3=45 configs; `main()` argparse entrypoint.
- `$Config = (velocity_thresh, merge_gap, min_window_duration)`; const
`BASELINE=(0.02,2.0,2.0)`. **`load_golden_gts` + `make_predict_fn` + `BASELINE` are imported by
4 downstream drivers** (calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
- ? grid is selected on full GT ? `improvement_report` here is an **optimistic upper bound**;
trustworthy number is the cross-val held-out recall.
- `@backend/eval/calibrate_local.py` ? distill Gemini?local knobs (windowing + net-crossing
gate, no cloud at runtime). `make_local_predict_fn(...)` adds `rally_gate.apply_gate` when
`min_crossings>0`; `local_grid()` 5x3x3x4=180 configs; `build_videos(specs)`;
`run(specs,iou=0.5,metric="f1")` reports per-video best-fit (optimistic) + `leave_one_video_out`
(honest); `main()` `--manifest`.
- `$LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; const
`BASELINE_LOCAL=(0.02,2.0,2.0,0)`.
- `$manifest JSON: [{name, trajectory, fps, frame_width, labels}, ...]` (`labels`=Gemini silver
CSV) ? **shared with `distill_local`**.
- ? `build_videos` raises `SystemExit` if any labels file yields no rallies.
- `@backend/eval/export_wasb_segments.py` ? turn a tuned config into eyeball-able segments +
golden comparison + optional clip cut.
`run(trajecory_csv, fps, frame_width, cfg=TUNED, ..., min_crossings=0)`;
`segments_to_rows`/`write_segments_csv` ($golden/slicer schema
`SEG_FIELDS=[rally_number, start_time, end_time, duration, start_mmss, end_mmss]`);
`build_comparison(preds,gts,iou=0.5)` golden-centric per-rally matched/MISS/false_positive rows;
`write_comparison_csv` ($`COMP_FIELDS`); `seconds_to_mmss(t)`; `_maybe_slice` lazy-imports
`backend.tools.rally_slicer.slice_rallies`; `main()`.
- `$Config=(velocity, merge_gap, min_dur)`; const `TUNED=(0.05,1.0,1.0)` (F1 0.588?0.746). ?
`write_segments_csv` output is consumed by `rally_slicer.load_rally_labels` AND re-imported by
`gemini_refine` ? schema is load-bearing across modules. ? inner `_gate` re-parses trajectory
only when `min_crossings>0`.
```

DB-backed scoring + corpus aggregation + regression

```
- `@backend/eval/harness.py` ? score stored DB predictions (no pipeline re-run,
free/deterministic). `get_predictions(db, video_id, stage, detector=None)` reads
`candidates` ? `query_candidate_segments` / `final` ? `query_play_segments`;
`evaluate(db, video_id, gt_intervals, stages, iou=0.5, detector=None, with_pr_curve=False)`;
`report_to_dict`; `format_report_text(report, show_failures=False)`; `_rows_to_intervals`,
`_fmt_ts`.
- `$STAGES=("candidates","final")` ? candidates=raw detector windows, final=AI-confirmed;
diffing the two localizes detection-vs-segmentation failure.
`StageReport` (stage, pred_intervals, score, match_result, pr_curve);
`EvalReport` (video_id, iou_threshold, gt_intervals, stages). ? `get_predictions` raises
`ValueError` on unknown stage. **`get_predictions` is reused by `regression.py`**.
- `@backend/eval/batch.py` ? pure corpus-aggregation pieces (testable without GPU/DB/API;
```

orchestration lives in `run\_phase3\_eval.py`, not here).

```

`resolve_video_path(local_path,videos_root)` normalizes collector `./data\...` Windows paths;
`default_stages_for(segmenter,requested="auto")`; `aggregate(per_video,stages)` micro P/R/F1
(pooled TP/FP/FN) + TP-weighted mean IoU + macro-F1; `format_aggregate(agg)`.
- ? `_FINAL_ONLY_SEGMENTERS={"motion","gemini"}` ? segmenters with no candidate stage; `auto`
scores both stages for two-stage detectors (yolo_hybrid/tracknet_*) else `final` only.
$ aggregate` consumes `report_to_dict`-shaped dicts (`stages[stage]["score"]`). ? `_safe_div`
returns 0.0 on zero denominator (no div-by-zero).
- `@backend/eval/regression.py` ? regression runner vs snapshotted "known-good" baseline.
`regress(db,video_id,ground_truth,stage="final",iou=0.5,...,tolerance=0.05,baseline_path)`;
`regress_with_provider(db,video_id,video_ref,tier="file",...)`. pulls GT from
?`annotation_registry.get(tier)` (`backend.annotations`); `load_baselines`/`save_baseline`;
`format_result`.
- $`RegressionResult` dataclass (video_id,stage,iou_threshold,precision,recall,f1,baseline_pre
cision,baseline_recall,tolerance,regressed,reasons; `.as_dict()`). $baseline store: JSON keyed
`"{video_id}::{stage}"`, `DEFAULT_BASELINE_PATH=output/regression_baseline.json`,
`DEFAULT_TOLERANCE=0.05`.
- ? `tier`?provider mapping: "file"?FileProvider (CI), "vendor"?HumanVendorProvider,
"auto"?oracle. ? no baseline ? `regressed=False` (nothing to compare). ? regression flagged when
**precision OR recall** drops > tolerance (F1 not gated). ? re-baseline only after a
*legitimate* improvement or you suppress real regressions.

```

Learned local segmenter (ADR-004: offline, API-free)

```

- `@backend/eval/training.py` ? pure data layer for the learned classifier.
`label_candidates(candidate_intervals,gt_intervals,iou=0.5)` labels via the **same
`match_intervals`** the evaluator uses (train/score consistency); `extract_yolo_features(row)`;
`build_training_set(candidate_rows,gt_intervals,iou=0.5,feature_fn=extract_yolo_features)` ?
`(X,y,intervals)`.
- ? `feature_fn` is the substrate plug-in seam (YOLO today, WASB later supplies its own).
$`YOLO_FEATURE_NAMES=[duration,peak_velocity,mean_velocity,active_frame_count,track_id_count]`
read from `candidate_segments.stats` (JSON, populated by `HybridYoloSegmenter`). ? missing stats
fields default 0.0 (sparse/old rows still well-formed).
- `@backend/eval/classifier.py` ? dependency-free `LogisticRegression` (numpy only, no sklearn).
`fit(X,y,feature_names=None,class_weight="balanced")` standardizes + balanced-class GD;
`predict_proba`/`predict(threshold=0.5)`; `to_dict`/`from_dict`/`save`/`load` (small human-
readable JSON, no pickle). Ctor: `lr=0.1,epochs=1000,l2=1e-3`.
- ? `_sigmoid` clips z to [-30,30]; constant features guarded (`std==0 ? 1.0`). ? stores
feature standardization (`mean`/`std`) in the model ? inference MUST use same feature order.
$JSON contract has `"model":"logreg"` tag.
- `@backend/eval/distill_local.py` ? distill Gemini ? a **learned** classifier (goes beyond
`calibrate_local`'s threshold ceiling). `build_candidates(...)` recall-oriented proposal +
fps/resolution-INVARIANT features; `baseline_windows`; `load_video(spec,iou=0.5)`;
`predict_rallies(model,X,intervals,threshold=0.5)` filters by proba then
`gemini_refine.merge_overlaps(gap_tol=1.0)`; `run(specs,iou,threshold,model_out)` leave-one-
video-out learned-vs-baseline + final all-videos model; `main()`.
- $`FEATURE_NAMES=[duration,peak_velocity,mean_velocity,active_ratio,net_crossings]`
(deliberately fps/res-invariant for cross-video transfer); consts `PROPOSAL=(0.005,1.0,0.5)`
(permissive recall-first), `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? `net_crossings` via
`rally_gate.gate_windows(...,min_crossings=0)` (`crossings` count). ? leave-one-video-out only
runs with ?2 videos; final model always trained on all.

```

Gemini refinement driver (eval/tuning, standalone ? not the live pipeline)

```

- `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips
(precision lever; over-fires on service feeds otherwise).
`merge_overlaps(intervals,gap_tol=0.0)` (gap_tol heals shuttle-invisibility splits without
merging real rallies ? **reused by `distill_local`**); `refine_window(...)` cuts+downscales
clip, fresh provider per call; `run(video,trajectory,...,cfg=BASELINE,pad=3.0,model="gemini-
flash-latest",max_workers=4,...)`; `full_video_rallies(...)` full-context chunked pass (recall
not bounded by WASB candidates); `offset_segments`; `main()` (`--full-video` skips
`--trajectory`).
- ? `provider_registry.get("gemini",...)` + `sport_registry.get("badminton")`. ? **fresh genai
client per worker** ? shared client across threads throws socket errors (not thread-safe). ?
keeps Gemini's per-clip segments separate (collapsing to one envelope swept dead time into
"rallies" = precision loss); tiny splits healed by the global `gap_tol=1.0` merge. ? requires
`GEMINI_API_KEY` env (raises `SystemExit`). ? imports `BASELINE` from calibrate_wasb and

```

`TUNED`/`write\_segments\_csv`/`seconds\_to\_mmss` from export_wasb_segments.

Audio probe (diagnostic only)

- `@backend/eval/audio/e1\_probe.py` ? E1 GO/NO-GO audio probe (docs/AUDIO_RALLY_DETECTION_PLAN.md); no training/fusion, purely diagnostic.
`cluster\_hits(hits,gap\_sec=2.0,min\_hits=2,pad=0.3)` groups thwacks into rally proposals;
`run(video,labels\_csv,trajectory,fps,frame\_width,k=2.5,...)` reports (1) discrimination (in-rally vs dead-time hit density ratio), (2) recall recovery of WASB-missed rallies, (3) audio-only proposal P/R/F1; `\_hits\_in`; `main()`.
- reuses `pipeline.audio.hit\_detector.detect\_hits\_in\_video/HitEvent` +
`calibrate\_wasb.{load\_golden\_gts,make\_predict\_fn,BASELINE}` + `metrics.{match\_intervals,score}`.
? outcome (per MEMORY/audio-e1-findings): recall 100% but discrimination only ~2.2x, band-pass null ? **audio NOT adopted for fusion** (PR #35). ? `lone hit = service feed`, so `min\_hits>=2` to count as a rally.

Tests (`tests/`) ? one file per eval module

- `@tests/test\_eval\_metrics.py` ? metrics.py (IoU, matching, scoring, pr_curve)
- `@tests/test\_eval\_annotations.py` ? annotations.py (timestamp parsing, CSV load/validation)
- `@tests/test\_calibration.py` ? calibration.py (kfold, select_best, cross_validate, leave_one_video_out, improvement_report)
- `@tests/test\_calibrate\_wasb.py` ? calibrate_wasb.py (load_golden_gts, predict_fn, grid, run)
- `@tests/test\_calibrate\_local.py` ? calibrate_local.py (local grid + gate, leave-one-video-out)
- `@tests/test\_export\_wasb\_segments.py` ? export_wasb_segments.py (segment/comparison CSV schemas, mmss)
- `@tests/test\_eval\_harness.py` ? harness.py (stage prediction fetch, evaluate, report formatting)
- `@tests/test\_eval\_batch.py` ? batch.py (path resolution, stage selection, aggregate)
- `@tests/test\_eval\_regression.py` ? regression.py (baseline store, regress, provider wiring)
- `@tests/test\_eval\_training.py` ? training.py (label_candidates, feature extraction, build_training_set)
- `@tests/test\_eval\_classifier.py` ? classifier.py (fit/predict_proba, balanced weights, JSON round-trip)
- `@tests/test\_distill\_local.py` ? distill_local.py (candidate building, predict_rallies, learned-vs-baseline)
- `@tests/test\_gemini\_refine.py` ? gemini_refine.py (merge_overlaps, offset mapping, refine flow)
- Adjacent (consumed by eval but owned elsewhere): `@tests/test\_hit\_detector.py` (audio hit_detector used by e1_probe), `@tests/test\_rally\_gate.py` (rally_gate used by calibrate_local/distill_local/export), `@tests/test\_annotation\_providers.py` +
`@tests/test\_automated\_provider.py` (annotation_registry used by regression.py). No dedicated test file exists for `audio/e1\_probe.py` (diagnostic-only script).

Cross-module dependency notes (footguns when refactoring)

- ? `metrics.Interval`/`match\_intervals`/`score` are the spine ? imported by harness, calibration, training, distill_local, export, e1_probe.
- ? `calibrate\_wasb.{load\_golden\_gts, make\_predict\_fn, BASELINE}` is a de-facto shared API for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema (`start\_time`/`end\_time`) or `BASELINE` ripples widely.
- ? two distinct GT CSV schemas coexist: `annotations.py` (`start,end`) vs
`calibrate\_wasb.load\_golden\_gts` (`start\_time,end\_time`, also `export\_wasb\_segments` write schema) ? don't cross-feed.
- ? `gemini\_refine.merge\_overlaps` is reused by `distill\_local.predict\_rallies` (gap_tol=1.0);
`harness.get\_predictions` is reused by `regression.regress`.

chunk: Providers / sports / annotations / utils / tools

All files read. Producing the CODE_MAP chunk.

Subsystem: Providers / Sports / Annotations / Utils / Tools

Three parallel registry-based plugin subsystems (AI providers, sport profiles, annotation sources) plus stateless utility/tool modules. All three registries follow the same pattern: a `base.py` ABC + a registry instance auto-populated at import time.

AI Providers ? `backend/providers/` (LLM video-analysis backends)

```
- ? `@backend/providers/__init__.py::ProviderRegistry` ? registry keyed by string name ?
`Type[AIVideoProvider]`; module-level singleton
`@backend/providers/__init__.py::provider_registry`. Auto-registers `"gemini"` in `__init__`.
- `ProviderRegistry.register(name, provider_cls)` ? manual registration (NOT decorator-style;
differs from annotation registry).
- `ProviderRegistry.get(name, api_key, model_name)` ? instantiates with creds; raises
`KeyError` listing available names if missing.
- `ProviderRegistry.list_available()` ? `{name: docstring}`.
- `@backend/providers/base.py::AIVideoProvider` ? ABC; ctor takes `(api_key, model_name)`.
- $ `@backend/providers/base.py::AIVideoProvider.analyze_video(video_path, prompt,
chunk_duration=0.0, label="")` ? **`Tuple[list, dict]`** = `(segments_list, metrics_dict)`.
Metrics keys: `upload_time`, `process_time`, `gen_time`. Segment dicts shaped like
`{"start_time", "end_time", ...}`. This tuple contract is the cross-cutting data shape every
provider must honor.
- `@backend/providers/gemini.py::GeminiProvider` ? Google `genai` impl (`from google import
genai`).
- `@backend/providers/gemini.py::GeminiProvider.client` ? lazy `@property` building
`genai.Client(api_key=...)` on first use (avoids auth at instantiation).
- $ const `@backend/providers/gemini.py::MAX_UPLOAD_MB` = `500` ? soft cap (real Gemini File
API limit is 2GB; capped lower for safety/speed).
- `GeminiProvider.analyze_video(...)` ? pipeline: size-check ? upload (3 retries) ? poll
`PROCESSING` state (10s loop) ? `generate_content` (3 retries,
`response_mime_type="application/json"`, `temperature=0.0`) ? delete remote file ? `json.loads`.
- ? On ANY failure (oversize, upload fail, processing FAILED, status check, empty response,
non-array JSON, parse error) returns `([], metrics)` ? **empty list, never raises**. Callers
cannot distinguish "no rallies" from "provider failed" by return value alone (metrics still
populated).
- ? Requires `ffprobe`/`network`/`GEMINI` access; uploaded file is always deleted in a best-
effort `try/except` (swallowed) ? orphaned remote files possible if process dies mid-run.
- ? Caller MUST pre-chunk so each clip < `MAX_UPLOAD_MB`; oversize clips are silently skipped
(logged error + empty return).
```

Sports ? `backend/sports/` (per-sport prompts + relevance config)

```
- ? `@backend/sports/__init__.py::SportRegistry` ? registry keyed by `profile.name` ?
`Type[SportProfile]`; singleton `@backend/sports/__init__.py::sport_registry`. Auto-registers
`BadmintonSportProfile`.
- ? `SportRegistry.register(profile_cls)` **instantiates** the profile (`profile_cls()`) just
to read `.name` for the key ? profiles must be cheaply constructible with no args.
- `SportRegistry.get(name)` ? fresh instance; `KeyError` lists available if missing.
`list_available()` ? `{name: docstring}`.
- `@backend/sports/base.py::SportProfile` ? ABC. Abstract members: `name` (`@property`),
`get_prompt(chunk_duration=0.0)`, `get_relevance_config()`.
- `@backend/sports/badminton.py::BadmintonSportProfile` ? `name == "badminton"`.
- $ `BadmintonSportProfile.get_prompt(chunk_duration=0.0)` ? large hardcoded CV-extraction
system prompt. Output schema is a JSON array of `{start_time, end_time, shuttle_exchange_count,
shot_count_confidence}` where times are **TOTAL DECIMAL SECONDS** (heavily emphasized; rejects
MM:SS / concatenated formats). `shot_count_confidence` ? {Low, Medium, High}. When
`chunk_duration > 0.0`, prepends a context block clamping all timestamps to `[0,
chunk_duration]`.
- $ `BadmintonSportProfile.get_relevance_config()` ? `{"court_green_lower": [35,40,40],
"court_green_upper": [85,255,255], "court_pixels_min_ratio": 0.05}` ? HSV green-court thresholds
for downstream visual relevance validation.
- ? Prompt schema (`shuttle_exchange_count`/`shot_count_confidence`) differs from the provider
base's generic `{start_time,end_time}` contract ? provider returns raw model JSON; sport prompt
defines the richer keys.
```

Annotations ? `backend/annotations/` (pluggable ground-truth rally-label sources)

Golden-set vendoring layer (ADR-006, `docs/GOLDEN\_SET\_VENDORING.md`). Mirrors the provider registry pattern but uses **decorator-friendly self-registration**.

```
- ? `@backend/annotations/__init__.py` ? re-exports `annotation_registry`, `AnnotationProvider`;
imports `file_provider` & `automated_provider` purely for their registration **side-effects**
(noqa F401). `__all__ = ["annotation_registry", "AnnotationProvider"]`.
- $ `@backend/annotations/base.py::Interval` = `Tuple[float, float]` ? **`(start, end) seconds`
identical to `backend.eval.metrics.Interval`**. This symmetry is the whole point: ground truth
```

from file/vendor/oracle all feed the eval harness unchanged.

- `$` Status consts `@backend/annotations/base.py::QUEUED`/`RUNNING`/`DONE`/`FAILED`` and tuple `STATUSES` = `("queued","running","done","failed")``.
- `@backend/annotations/base.py::AnnotationProvider`` ? ABC, `**submit` ? poll` ? fetch**` lifecycle (supports async vendors).
 - `$`name`` (`@property``, registry key), `submit(video_ref, guideline=None, sport="badminton")`` ? `job_id:str``, `poll(job_id)`` ? `status:str``, `fetch(job_id)`` ? `List[Interval]`` (sorted; ? should `**raise**` if not DONE rather than return `[]``, which would silently skew scores).
 - `AnnotationProvider.annotate(video_ref, guideline=None, sport="badminton")`` ? convenience `submit+poll+fetch` for synchronous providers; ? `**raises`RuntimeError**` if job isn't immediately `DONE`` (async providers must drive `submit/poll/fetch` manually).
 - ? `@backend/annotations/base.py::AnnotationProviderRegistry`` ? keyed by provider ``name``; singleton `@backend/annotations/base.py::annotation_registry``.
 - `register(provider_cls)`` ? returns the class (`**decorator-friendly**`); keys by `provider_cls().name``, ? falling back to `__name__`` if the no-arg ctor throws (providers needing ctor args register under their class name, not their `.name``).
 - `get(name, **kwargs)`` ? instantiates passing through ctor kwargs (e.g. `labeler=`, lineage=)``; `KeyError`` lists available. `list_available()`` ? `List[str]`` (note: list of names, NOT the `{name:doc}`` dict the provider/sport registries return).
- `@backend/annotations/file_provider.py::FileProvider`` ? `name == "file"``. Synchronous, zero-cost default; the "job" is a CSV already on disk (same `start,end`` format as `run_eval.py --scaffold``). Self-registers via `annotation_registry.register(FileProvider)`` at bottom.
 - ctor `FileProvider(csv_dir=None, ext=".csv")``. `_resolve(video_ref)``: existing path used as-is, else `<csv_dir>/<video_ref><ext>`` (key by `video_id/MD5``). `submit`` returns the resolved path `**as the job_id**`; `poll`` ? `DONE`/`FAILED`` on `os.path.isfile``; `fetch`` delegates to `@backend/eval/annotations::load_annotations`` (raises `FileNotFoundError`` if missing).
 - ? `guideline`/`sport`` args are ignored (a file's labels already exist).
- `@backend/annotations/automated_provider.py::AutomatedProvider`` ? `name == "auto"``. `**Interface stub only**` (GS-4); no concrete model wired. Self-registers at bottom.
 - `$`type` alias @backend/annotations/automated_provider.py::Labeler` = `Callable[[str, Optional[str], str], List[Interval]]` (= `(video_ref, guideline, sport)` ? intervals`).`
 - ctor `AutomatedProvider(labeler=None, lineage="unspecified")`` ? defaults `labeler`` to `_not_configured``; in-process synchronous job store `_jobs: Dict[job_id, List[Interval]]``. `submit`` runs `labeler`` now, sorts `(float,float)`` intervals, stashes under `job_id = video_ref``. `poll`/`fetch`` read `_jobs``.
 - `@backend/annotations/automated_provider.py::_not_configured(...)`` ? default `labeler`` that `**raises`NotImplementedError**` (keeps provider constructible for registry keying while failing loudly on accidental use; points to `docs/BACKLOG.md`` GS-4).
 - ? ORACLE RULE ?
- `@backend/annotations/automated_provider.py::warn_if_same_lineage(oracle_lineage, production_lineage)`` ? `bool``; logs a warning + returns `True`` when oracle and production share a model lineage (shared blind spots make the regression test green-while-wrong). It only `**warns**`, does not block ? CI wiring must act on the `bool``. `lineage`` ctor arg records the oracle's family for this guard.
 - ? Registered under name `"auto"`` only because the no-arg ctor succeeds (`labeler`` defaults to the stub) ? see the registry's `__name__`` fallback caveat above.

Utils ? `backend/utils/` (stateless helpers)

- `@backend/utils/video.py`` ? `ffmpeg/ffprobe` wrappers (? require `ffmpeg`` + `ffprobe`` on PATH).
 - `@backend/utils/video.py::get_video_duration(video_path)`` ? float seconds via `ffprobe``; ? returns `0.0`` on any failure (logged) ? caller must treat `0.0`` as "unknown", not a real duration.
 - `@backend/utils/video.py::extract_video_chunk(video_path, start_time, duration, output_path, reencode=False, scale_width=None)`` ? `bool``. ? `reencode=False`` = fast stream-copy but cuts at `**nearest keyframe**` (boundary drift); `reencode=True`` (`libx264 ultrafast/crf23``) = frame-accurate. `scale_width`` requires `reencode=True`` (else ignored). Returns `False`` on failure; `ffmpeg`` stdout/stderr sent to `DEVNULL``.
- `@backend/utils/geometry.py`` ? pure bbox math, format `(x1, y1, x2, y2)``.
 - `get_center(bbox)``, `get_distance(p1, p2)`` (Euclidean), `get_velocity(bbox1, bbox2, fps)`` (px/s of center), `get_velocity_normalised(bbox1, bbox2, fps, frame_width)`` (fraction of frame-width/s ? resolution-independent thresholds; ? falls back to raw px velocity if `frame_width <= 0``).
 - ? `@backend/utils/geometry.py::calculate_iou(boxA, boxB)`` uses the `+1`` pixel-area convention (`(x2-x1+1)*(y2-y1+1)``); returns `0.0`` when intersection area is 0. Mixing with `+0`` IoU conventions elsewhere will skew thresholds.

Tools ? `backend/tools/` (standalone CLIs)

- `@backend/tools/rally\_slicer.py` ? CLI to cut padded per-rally clips from a long annotated video (offline verification / per-video calibration). Output naming: `<out_dir>/<video_stem>__rally_<n>.mp4` (rally id `` ? ``).
- \$ `consts @backend/tools/rally_slicer.py::BEGIN_PADDING = `0.5`, `END_PADDING` = `1.0` ? ?` deliberately match `VideoCompiler` defaults so sliced clips frame rallies identically to train/eval.
- \$ `@backend/tools/rally_slicer.py::ClipWindow` ? dataclass `(rally: str, start: float, end: float)`; `rally` kept as **string** to allow inserted ids like "7.5".`
- `@backend/tools/rally_slicer.py::load_rally_labels(csv_path)` ? `Dict[str, (start, end)]`.` Accepts golden schema `rally_number, start_time, end_time[, start_mmss, end_mmss]`; ? lowercases/strips headers, prefers decimal cols, falls back to mm:ss via `@backend/eval/annotations::parse_timestamp``; `**silently skips**` rows with missing/degenerate times (`end <= start``, e.g. unfilled MISSED rows).
- `@backend/tools/rally_slicer.py::compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, video_duration=None)` ? `List[ClipWindow]` ? **pure & unit-tested** window math (pad + clamp to duration). rally_numbers == ["all"]` ? every labeled rally in time order; unknown ids silently dropped; windows entirely past EOF or with `pe <= ps` skipped; rounds to 3 dp.`
- `@backend/tools/rally_slicer.py::_probe_duration(video_path)` ? `Optional[float]` (ffprobe; `None` on failure ? separate from utils/video.get_video_duration` which returns 0.0`).`
- `@backend/tools/rally_slicer.py::slice_rallies(video_path, labels_csv, rally_numbers, out_dir, begin_pad, end_pad)` ? `List[str]` written paths. The ONLY ffmpeg-touching fn (libx264 superfast/crf22 + aac 128k, -ss/-to` accurate cut). ? Raises FileNotFoundError`/`ValueError` for missing video / no usable rows / no matched rallies; per-rally ffmpeg failures are caught + printed + skipped (not fatal).`
- `@backend/tools/rally_slicer.py::main()`) ? argparse entry (--video --labels --rallies --out-dir --begin-pad --end-pad`); --rallies` is comma-separated or all`.`

Cross-cutting notes

- ? Three registry idioms diverge: providers/sports use `**manual `register() + `list_available()?dict`**`; annotations use `**self-registering decorator-friendly `register() returning the class + `list_available()?list`**`. Don't assume one shape.
- \$ The `(start, end)` / `Interval`` seconds tuple is the universal label currency tying `backend/annotations`, `backend/eval/metrics`, and `backend/eval/annotations.load_annotations`` together.
- External deps not in this subsystem but referenced: `@backend/eval/annotations::load_annotations` (FileProvider), @backend/eval/annotations::parse_timestamp` (rally_slicer).` System deps: `ffmpeg`/`ffprobe` (video.py, rally_slicer), `google.genai` (gemini).`
- Note: no `tests/`` directory was in the provided file list, so none enumerated; `compute_clip_windows`` and the geometry helpers are referenced as unit-tested but their test files were not in scope.

chunk: Tests (test -> subsystem map)

I now have all the test files read. Here is the dense CODE_MAP chunk for the Tests subsystem.

Tests (test ? subsystem map)

`tests/`` ? pytest suite. Pure-logic + mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked (all I/O boundaries patched). Each file maps to one production subsystem. \$marks data contracts the tests pin; ? marks registry/plugin extension points exercised; ? marks footguns the tests deliberately lock down.

- `@tests/__init__.py`` ? empty package marker (`# Tests package``).

Eval / metrics subsystem (`backend/eval/\*`)

- `@tests/test_eval_metrics.py`` ? covers `backend/eval/metrics`` (pure interval matching). Tests `interval_iou`, `match_intervals` (? MatchResult.matches`/`.unmatched_pred_indices`/`.unmatched_gt_indices`, `.matches[.pred_index`), `score` (? Score.precision/recall/f1/mean_iou/mean_start_error/mean_end_error/true_positives/false_positives/false_negatives`, `.as_dict()`), `pr_curve`. $score.as_dict()`) keys incl. iou_threshold`. ? greedy matcher prefers higher-IoU pred; one GT`

not double-claimed; default `pr\_curve` = 9 thresholds; degenerate/zero-width intervals ? IoU 0.

- `@tests/test\_eval\_batch.py` ? covers `backend/eval/batch`. `batch.resolve\_video\_path` (? normalizes `./data\..` Windows-style relative paths against a collector root; absolute kept; None/"" ? None), `batch.default\_stages\_for` (? `auto` ? `["final"]` for `motion`/`gemini`, `["candidates","final"]` for two-stage `yolo\_hybrid`; explicit overrides auto), `batch.aggregate` (\$per-video shape `{"stages":{"final":{"score":{"tp,fp,fn,mean\_iou,f1}}}}`); emits `micro\_precision/micro\_recall/micro\_f1`, TP-weighted `mean\_iou`, `macro\_f1`, `videos`; handles empty list / zero-division).
- `@tests/test\_eval\_classifier.py` ? covers `backend/eval/classifier::LogisticRegression` (numpy-only impl). `.fit(X,y,feature\_names=,class\_weight=)`, `.predict`, `.predict\_proba` (range [0,1]), `.save`/`.load` (\$JSON roundtrip preserves `feature\_names`), ? `predict\_proba` before fit raises RuntimeError; `fit` on empty raises ValueError; `class\_weight="balanced"` recovers 10:1 minority.
- `@tests/test\_eval\_training.py` ? covers `backend/eval/training`. `extract\_yolo\_features` (\$row shape `{start\_time,end\_time,stats:{peak\_velocity,mean\_velocity,active\_frame\_count,track\_id\_count}}` ? 5-vec; missing stats default 0), `YOLO\_FEATURE\_NAMES`, `label\_candidates` (IoU-threshold labeling, one-to-one best match), `build\_training\_set` (? `(X,y,intervals)`; `feature\_fn` override). ? feature order is `[dur,peak,mean,frames,tracks]`.
- `@tests/test\_eval\_annotations.py` ? covers `backend/eval/annotations`. `parse\_timestamp` (decimal/MM:SS`/`HH:MM:SS; ? raises on `"/"abc"/4-part`), `load\_annotations` (\$CSV `start,end`; header auto-detected, blank/`#`comment lines skipped, sorted by start; ? rejects start>end and short rows), `write\_scaffold` (header-only template loads to `[]`).
- `@tests/test\_eval\_metrics.py` + `@tests/test\_eval\_harness.py` are the keystone scorer; harness file ? covers `backend/eval/harness`: `evaluate` (? `Report.stages[stage].score`/`.pr\_curve`), `get\_predictions` (stages `final`/`candidates` w/ `detector=` filter; ? unknown stage raises ValueError), `format\_report\_text` (`show\_failures=` lists "missed rallies" w/ `HH:MM:SS`), `report\_to\_dict` (\$JSON-serialisable, `video\_id`/`stages.<s>.score.f1`), `\_rows\_to\_intervals` (sorts). Also exercises CLI `@run\_eval.py::main` (? exit codes: 0 ok, 2 missing-annotations / missing-db; `--scaffold`/`--json` flags) and `run\_eval.get\_file\_md5` (32-char stable md5).
- `@tests/test\_eval\_regression.py` ? covers `backend/eval/regression` (GS-2 regression runner) + CLI `@run\_regression.py::main`. `save\_baseline`/`load\_baselines` (\$baseline store keyed `"<vid>::<stage>"` ? `{precision,recall,f1}`; absent ? `{}`), `regress` (? `RegressionResult.regressed/.precision/.recall/.reasons`; ? "no baseline" is NOT a regression; recall drop beyond `tolerance` regresses w/ reason mentioning "recall"; default tol 0.05), `regress\_with\_provider` (tier="file" via GS-1 FileProvider). ? CLI exit-code CI gate: `--update-baseline` ? 0; regression ? 1, prints "REGRESSED".
- `@tests/test\_calibration.py` ? covers `backend/eval/calibration` (overfitting guard). `pct\_improvement` (? from-zero ? `inf`; 0?0 ? 0.0), `kfold\_indices` (? caps k to n; k=0 ? []; train/test disjoint & cover all), `select\_best` (picks perfect config by `metric`), `cross\_validate` (\$result `{metric,folds,mean\_test,generalization\_gap,picked\_configs}`; default `metric="recall"`; too-few ? `{error":...}`), `leave\_one\_video\_out` (\$video dict `{name,predict\_fn,gts}`; ? `{mean\_held\_out,per\_video[]}.config`; needs ?2), `improvement\_report` (? `{baseline,tuned,precision\_delta\_pts,precision\_delta\_pct}`).
- `@tests/test\_calibrate\_wasb.py` ? covers `backend/eval/calibrate\_wasb` (WASB calibration driver; no WSL/GPU). `load\_golden\_gts` (\$golden CSV `rally\_number,start\_time,end\_time,shots\_count`; ? incomplete rows skipped), `make\_predict\_fn` (trajectory CSV \$`Frame,Visibility,X,Y` ? moving-segment windows), `run` (? `{best\_cfg,baseline:{f1},cv:{mean\_test}}`), `BASELINE` const.
- `@tests/test\_calibrate\_local.py` ? covers `backend/eval/calibrate\_local` (local-only, no Gemini). `local\_grid` (? size = 5x3x3x4 = velocityxmergexmin_durxmin_crossings), `make\_local\_predict\_fn` (cfg tuple `(velocity,merge,min\_dur,min\_crossings)`; ? min_crossings gate drops monotonic motion).
- `@tests/test\_distill\_local.py` ? covers `backend/eval/distill\_local` (distilled local classifier). `build\_candidates` (? `(intervals,X)`; \$`FEATURE\_NAMES` = 5 invariant features, duration>0), `predict\_rallies` (`threshold=` cutoff; ? 1.01 ? []; 0.0 ? merged windows). Imports `backend/eval/classifier::LogisticRegression`.
- `@tests/test\_export\_wasb\_segments.py` ? covers `backend/eval/export\_wasb\_segments` (eyeball-verification export). `seconds\_to\_mmss` (? negative clamps to "0:00.000"), `segments\_to\_rows` (\$row `{rally\_number,start\_time,end\_time,duration,start\_mmss}`, time-ordered numbering), `write\_segments\_csv`, `build\_comparison` (? `{score,rows[type?{golden,pred\_extra}],num\_gt}`; statuses `matched`/`MISS`/`false\_positive`), `run` (`min\_crossings` gate, `baseline=`). ? keystone reuse claim: exported CSV loads straight into `backend/tools/rally\_slicer::load\_rally\_labels`.
- `@tests/test\_gemini\_refine.py` ? covers `backend/eval/gemini\_refine` (pure helpers, no API).

`merge\_overlaps` (`gap\_tol=` stitches small gaps, preserves real ones), `\_offset\_segments` (\$input dicts `{start\_time,end\_time}` ? clip-offset tuples; ? end?start & unparseable dropped; clamps to clip bounds).

Audio subsystem (`backend/pipeline/audio/\*`, `backend/eval/audio/\*`)

- `@tests/test\_hit\_detector.py` ? covers `backend/pipeline/audio/hit\_detector` (pure DSP).
`detect\_hits` (? `HitEvent.t`; `k=` threshold, `band=` bandpass ? ? broadband clicks survive band-limit; silence ? []), `onset\_envelope` (? `(env, hop\_sec)`; band changes flux, same length), `\_moving\_stats`. Also `backend/eval/audio/e1\_probe::cluster\_hits`
(`gap\_sec`=`/min\_hits`; ? lone hits dropped, cluster spanned+padding). \$`HitEvent(t, <strength>`).

Detectors subsystem (`backend/pipeline/detectors/\*`)

- `@tests/test\_tracknet\_runner.py` ? covers `backend/pipeline/detectors/tracknet\_runner` (pure-Python pieces; WSL subprocess mocked via `\_wsl\_bash`). `\_to\_wsl\_mnt\_path` (? `C:\..` ? `/mnt/c/..`; POSIX/~` passthrough), `TrackNetConfig.from_indexing_cfg` (\$indexing `{tracknet:wsl_distro,conda_env,weights_path,queue_length,stage_in_wsl}`; defaults `distro=Ubuntu, env=TrackNetV4, weights=""` must be user-supplied),
`TrajectoryPoint(frame,visible,x,y)`, `TrackNetRunner.parse\_trajectory\_csv`
(\$`Frame,Visibility,X,Y`; visibility 0?False), `\_trajectory\_to\_action\_windows` (\$window `{start,end,active\_frame\_count,peak\_velocity,chunk\_index}`;
`velocity\_thresh`=`/merge\_gap`=`min\_window\_duration`; splits on invisible gap, ignores stationary), `\_healthcheck` (? missing wsl binary ? msg contains "wsl"), `\_run\_predict` (? requires weights, rejects non-`.mp4`, exit-0-but-no-CSV ? None), `\_stage\_video` (? `~/clips/<name>`), `stage\_in\_wsl=True` stages first.
- `@tests/test\_rally\_gate.py` ? covers `backend/pipeline/detectors/rally\_gate` (net-crossing gate; pure). `\_estimate\_play\_axis` (? `(axis,net,span)`; picks larger-spread axis),
`count\_net\_crossings` (counts sign changes across `net`; ? `deadband` ignores jitter; invisible points skipped), `gate\_windows` (? `[GateResult.kept/.crossings]`), `apply\_gate` (? filtered windows; `min\_crossings=`). ? kills single-toss (1 crossing), keeps multi-crossing rally.
Imports `TrajectoryPoint` from tracknet_runner.
- `@tests/test\_wasb\_cache.py` ? covers `backend/pipeline/detectors/wasb\_infer` (resumable cache layer; pure-Python). `\_det\_plain`/_`dets\_to\_tracker` (\$det `{xy:ndarray,score,scale}` ? plain `[x,y,score,scale]`; ? back-conversion restores ndarray for vector math), `\_load\_and\_clean\_cache` (? `(windows\_done,det)`; accumulates across windows in order; ? truncated trailing JSONL line recovered+rewritten to clean prefix; non-contiguous window index stops replay),
`\_cache\_paths`/_`append\_windows`/_`write\_manifest`/_`read\_manifest` (? atomic, no `.tmp` left; missing ? None), `manifest\_compatible` (\$manifest `{version,frames\_dir,weights,sport,frames\_in,frames\_total,windows\_total,windows\_done}`; ? invalidated by frames_total/weights/sport mismatch, empty, or `version` bump vs `CACHE\_VERSION`), `reset\_cache`, `default\_cache\_dir` (? `<stem>wasbcache` next to output),
`\_atomic\_write\_trajectory` (\$writes `Frame,Visibility,X,Y` header).

Segmenters subsystem (`backend/pipeline/segmenters/\*`)

- `@tests/test\_base.py` ? covers `backend/pipeline/segmenters/base`. ?
`SegmenterRegistry.register/get/list\_available` (? name?description map); `BasePlaySegmenter` ABC (`name`/_`description` props,
`\_process\_video(video\_id,video\_path,player\_pool=max\_frames=)`). Uses local `DummySegmenter`.
- `@tests/test\_gemini.py` ? covers `backend/pipeline/segmenters/gemini::GeminiPlaySegmenter` (provider_registry + video utils mocked). \$indexing `{chunk\_duration,chunk\_overlap}`. ? large chunk_duration ? single-file path; small ? chunked w/ per-chunk offset (chunk-2 local 30s ? global 170s); \$segment metadata `shot\_count` ? provider `shuttle\_exchange\_count`.
- `@tests/test\_yolo\_hybrid.py` ? covers
`backend/pipeline/segmenters/yolo\_hybrid::HybridYoloSegmenter` (Stage-1 detect/track + Gemini Stage-2; all mocked). `\_process\_video`, `\_extract\_action\_windows\_from\_chunk` (\$window `{start,end,active\_frame\_count,peak\_velocity,mean\_velocity,track\_id\_count,chunk\_index}`; offset into full-video coords), `\_detect\_and\_track` (? COCO `\_PERSON\_CLASS\_ID` filter ? "YOLO-was-0" gotcha; score threshold; returns `(track\_id,bbox)`). ? no GEMINI_API_KEY ? []; unparseable timestamps dropped; `store\_yolo\_candidates=False` skips DB persist+link; on success links candidate?segment (`link\_candidate\_to\_segment(cand\_id, seg\_id)`), candidate
`detector="yolo\_hybrid"`, `detection\_params.frame\_skip`.
- `@tests/test\_tracknet\_hybrid.py` ? covers
`backend/pipeline/segmenters/tracknet\_hybrid::TrackNetHybridSegmenter` (TrackNetRunner + provider mocked). ? `\_patched` decorator stacks 6 patches ? innermost-first injection order

(gotcha documented inline). `\_probe\_fps\_width` (? fallback `(30.0,1280.0)` when capture fails),
`process\_video` (happy path links candidate `detector="tracknet\_hybrid"` ? segment; no API key ?
[]; healthcheck failure aborts before `run\_predict`; no trajectory ? [];
`store\_yolo\_candidates=False` skips persist; drops unparseable segments), `.name`/`.description`
("TrackNet").

Providers subsystem (`backend/providers/\*`)

- `@tests/test\_providers.py` ? covers `backend/providers`. ?
`provider\_registry.list\_available/get(name, api\_key=, model\_name=)` (? instance);
`backend/providers/base::AIVideoProvider` ABC; `gemini` registered ? `GeminiProvider`.
- `@tests/test\_provider\_gemini.py` ? covers `backend/providers/gemini::GeminiProvider` (`genai`
client mocked). `analyze\_video` (? `(segments, metrics{upload\_time,process\_time,gen\_time})`;
`\$expects` JSON list `[{start\_time,end\_time}]`. ? non-list JSON ? []; ? rejects >`MAX\_UPLOAD\_MB`
(500) before upload (`upload.assert\_not\_called`); upload failure ? []; always
`files.delete(name=...)`. cleanup.

Annotations subsystem (`backend/annotations/\*`)

- `@tests/test\_annotation\_providers.py` ? covers `backend/annotations` + `file\_provider`. ?
`annotation\_registry.list\_available/get(name, \*\*kwargs)` (? unknown ? `KeyError`);
`AnnotationProvider` ABC; `file\_provider::FileProvider` (\$submit/poll/fetch contract;
`poll`?`DONE`/`FAILED` from `base`; bare video_id resolved to ``FAILED` + fetch raises `FileNotFoundError`; `.annotate()` convenience). ? keystone contract test:
provider `fetch()` output is `List[(float,float)]` directly scorable by
`backend/eval/metrics::score` (product/vendor/annotator graded by one scorer). ? `annotate()` on
missing file must fail loud, not return [].
- `@tests/test\_automated\_provider.py` ? covers
`backend/annotations/automated\_provider::AutomatedProvider` (GS-4 oracle stub). ? registered as
`"auto"`; `get("auto", labeler=, lineage=)` injects labeler (? provider sorts unsorted labeler
output by start). submit/poll/fetch (unknown job ? `FAILED` / `KeyError`). ? unconfigured labeler ?
`NotImplementedError` (fail loud). `warn\_if\_same\_lineage(a,b)` (? same lineage ? True + logs
"blind"/"lineage" warning; distinct/None ? False). End-to-end through
`backend/eval/regression::regress_with_provider(tier="auto",
provider_kwargs={labeler,lineage})`.

Validators subsystem (`backend/pipeline/validators/\*`)

- `@tests/test\_validators\_base.py` ? covers
`backend/pipeline/validators/base::ValidationRegistry`. ?
`register/run\_all/run\_all\_with\_context` (? `run\_all\_with\_context` is the obs-reporting addition:
returns `(results, shared\_context)` so producer validator can stash `ffprobe\_meta` for a
consumer; `run\_all` unchanged returns only results).
`\$ValidationResult(validator\_name,passed,message)`; `VideoValidator` ABC
(`.name`/`.description`/`.validate(video\_path,config,context)`). ? validator exception captured
(passed=False, message contains the error), not raised.

Config subsystem (`backend/config`)

- `@tests/test\_config.py` ? covers `backend/config` (Pydantic typed config). `load\_config` (?
nonexistent path ? model built-in defaults), `save\_default\_config`, `AppConfig`. \$defaults
pinned: `sport="badminton"`, `indexing.default\_segmenter="yolo\_hybrid"`,
`validation.min\_blur\_threshold=20.0`, `indexing.ai\_max\_workers=5`,
`indexing.tracknet.velocity\_threshold=0.02`, `ai\_provider="gemini"`,
`validation.min\_resolution\_width=1280`, `validation.relevance.court\_pixels\_min\_ratio=0.05`,
`stitching.begin\_padding=0.5`. ? extra/future keys tolerated; ? `AppConfig.get()` dict-compat
must return nested sections as plain dicts (legacy chained `.get("validation",{ }).get(...)`) via
dynamic section scan, not hardcoded list.

Infrastructure subsystem (`backend/infrastructure/database`)

- `@tests/test\_database.py` ? covers `backend/infrastructure/database::Database`. `\_\_init\_\_`
(creates `videos`/`play\_segments`; ? idempotent + upgrades pre-v1 DB to `candidate\_segments`;
`PRAGMA user\_version == 1`), `add\_video`/`get\_video`/`clear\_video\_data`,
`add\_play\_segment`/`query\_play\_segments`/`add\_event`, candidate API
`add\_candidate\_segment`/`query_candidate_segments(detector=,
only_unlabeled=)`/`link_candidate_to_segment`. \$`EXPECTED_CANDIDATE_COLUMNS` schema-contract
guard (? any column change must update this set ? id PK, NOT NULL on
video_id/detector/start_time/end_time/duration). ? candidate cascade-deletes on video delete;

`only\_unlabeled` excludes rows once `human\_label` set; `\_get\_connection`.

Sports profile subsystem (`backend/sports/\*`)

- `@tests/test\_sport\_profiles.py` ? covers `backend/sports`. ?
`sport\_registry.list\_available/get("badminton")` ? `BadmintonSportProfile`; `base::SportProfile`
ABC. `.name`, `.get\_prompt(chunk\_duration=)` (\$must contain "badminton"/"decimal seconds"/the
chunk value), `.get\_relevance\_config()` (\$keys
`court\_green\_lower/court\_green\_upper/court\_pixels\_min\_ratio`).

Tools subsystem (`backend/tools/\*`)

- `@tests/test\_rally\_slicer.py` ? covers `backend/tools/rally\_slicer` (pure logic, no ffmpeg).
`load\_rally\_labels` (\$CSV `rally\_number,start\_time,end\_time,start\_mmss,end\_mmss`; decimal or
`m:ss` fallback; ? MISSED/no-end rows & zero-duration skipped; fractional IDs like "16.5"
preserved), `compute\_clip\_windows` (? `[ClipWindow(rally,start,end)]`;
`BEGIN\_PADDING`/`END\_PADDING` consts; ? clamps to `video\_duration`; `"all"` orders by start-time
not dict order; unknown IDs dropped; window entirely past end skipped).

Reporting / observability subsystem (`backend/reporting/\*`, external `obsreport` lib)

- `@tests/test\_reporting\_harvest.py` ? covers `backend/reporting` (`emit\_indexer\_report`,
`harvest.record\_run`). ? `pytest.importorskip("obsreport")` ? soft dep, skips if absent.
\$`harvest.record\_run(...)` ? `RunRecord.envelope` (fields:
`verdict`{pass,pass_with_warnings,fail}`, `.flags[]` .code/.faq\_ref`,
`.video\_meta.fps/.fps\_source`{probed,fallback}`,
`.get\_stage(name).status/.metrics[]` .key/.value`, `.lint()`). ? footgun rules under test:
`silent\_provider\_noop` (no GEMINI_API_KEY + 0 segments, faq_ref `README#Q7`),
`ai\_empty\_vs\_no\_rallies` (key present, 0 segments ? warn), `synthetic\_motion\_metadata` (motion
segmenter; ? provider-noop must NOT fire for non-AI motion), `segmenter\_divergence` (fires only
when no explicit override), `reingest\_replaced\_prior`, `validation\_failed` (halts segmentation),
`fps\_fallback\_used`. `emit\_indexer\_report` writes `.report.md`/`.summary.md`/json next to
video; ? accepts plain dict OR typed `AppConfig` (normalized); `report.enabled=False` ? returns
None.
- `@tests/test\_reporting\_spec\_faq.py` ? covers `backend/reporting/spec.yaml` ? `README.md`
cross-reference integrity (golden guard). ? `importorskip("obsreport")`; uses
`obsreport.load\_spec`. `test\_every\_rule\_faq\_ref\_resolves\_to\_a\_readme\_heading` (? every
`rule.faq\_ref` "README#Qn" anchor must resolve to a real README heading ? Q-style prefix or
GitHub slug; locks "no dangling anchors"), `test\_actionable\_rules\_have\_faq\_refs` (? severity
`error`/`warn` rules must cite a faq_ref; `info` may omit).
\$`spec.rules[]` .code/.faq_ref/.severity`.

Utils subsystem (`backend/utls/\*`)

- `@tests/test\_geometry.py` ? covers `backend/utls/geometry`. `get\_center`, `get\_distance`,
`get\_velocity(bbox1,bbox2,fps)`, `get\_velocity\_normalised(...,frame\_width)` (? width?0 ? falls
back to raw velocity), `calculate\_iou` (? uses +1 bound expansion ? comment flags it;
identical?1.0, disjoint?0.0).
- `@tests/test\_video.py` ? covers `backend/utls/video` (subprocess mocked).
`get\_video\_duration` (? ffmpeg exception ? 0.0), `extract\_video\_chunk(in,start,dur,out)` (?
ffmpeg exception ? False; success ? True, `subprocess.run` called once).

Cross-cutting notes / footguns

- ? Two stale `.pyc` files in `tests/\_\_pycache\_\_` have no live source: `test\_gemini\_api.\*` and
`test\_geometry`/`test\_yolo\_hybrid` cpython-311 variants ? the current sources are the `.py`
files listed above (no `test\_gemini\_api.py` exists).
- ? Soft-dep gate: only the two `test\_reporting.\*` files import `obsreport` (skipped if not
installed); everything else runs with stdlib + numpy/cv2 only.
- ? CLI entrypoints are tested at repo root, not under `tests/`:
`@run\_eval.py::main`/`get\_file\_md5` (via test_eval_harness) and `@run\_regression.py::main` (via
test_eval_regression) ? both assert exit-code contracts for CI gating.
- Keystone "one-scorer" contract is asserted in three places that all feed
`backend/eval/metrics::score`: `@tests/test\_annotation\_providers.py` (provider output shape),
`@tests/test\_export\_wasb\_segments.py` (export? slicer reuse), `@tests/test\_eval\_harness.py` (DB
rows?intervals).

I'll start by reading the existing CODE_MAP.md to match its structure, legend, and density.

3. user (2026-06-07T09:12:51.819Z)

```
1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack.
4
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change.
6
7      **LEGEND**
8      - `@path` = file (always repo-relative)
9      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10     - `->` = dataflow / call / "produces"
11     - `$` = data contract (canonical shape)
12     - `?` = gotcha / footgun
13     - `?` = registry / plugin extension point
14     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16     ---
17
18     ## 1. PIPELINE
19
20     ### 1A. Runtime: video file -> highlight reel
21     ```
22     config.json (json.load; absent -> {})
23     -> video_id = MD5(whole file) @backend/main.py::get_file_md5 (? chunk size
varies per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE
file so values agree)
24     -> validate
@backend/pipeline/validators/base.py::registry.run_all(video_path, config)
25     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
26     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
27     -> seg = segmenter_registry.get(name)(db, config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry
28     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
29     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
30     -> select segment ids -> [(start,end)] (+ stitching padding)
31     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
32     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
33     ```
34     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
35     ```
36     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
37     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
38     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
39     -> P4 db.add_play_segment + db.link_candidate_to_segment
40     ```
41     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
42     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
43
```

```

44     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
45     ```
46     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
47     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
48     -> WasbRunner.run_predict(video, out_dir)
49     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
50     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
51     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
52     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
53     ```
54     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
55
56     ### 1C. Calibration / eval flow (NO pipeline run unless told)
57     ```
58     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
59     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
60     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
61     -> harness.evaluate -> EvalReport -> report_to_dict
62     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
63     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
64     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
65     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
66     ```
67     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
68
69     ---
70
71     ## 2. SUBSYSTEMS
72
73     ### 2.0 Orchestration & config
74     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). The
real app is under api/.
75     - `::app` FastAPI; `::db_instance`, `::global_config` module globals set ONLY in
`main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
76     - `::process_video` `@POST /api/process` -> validate -> add_video -> segment.
`::compile_highlights` `@POST /api/compile`. `::run_cli_diagnose_clip` lazy `import cv2`,
samples ±3s vs `indexing.motion_threshold`.
77     - ? uvicorn host hardcoded `127.0.0.1`; CLI default_segmenter fallback `motion`.
78     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), `default_segmenter` fallback `yolo_hybrid`, skips validation.
79     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
80     - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label
`collector_video_id`).
81     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
`*0`=ok, 1=REGRESSION (CI gate), 2=missing DB*. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run).
82     - `@config.json` ? single config. Sections: `validation` (+ `relevance`), `indexing` (+

```

```

.tracknet`; ? NO `wasb` block in checked-in config ? WASB runs use `WasbConfig` defaults),
`output`, `stitching`, root `sport/ai_provider/ai_model`. ? **three default segmenters
diverge**: config ships `gemini`, setup.py writes `yolo_hybrid`, main.py fallback `motion`.
83 - `@setup.py` ? write default config (skips if present), ffmpeg/torch/CUDA diagnostics,
GPU-aware pip (`nvidia-smi` -> cu124 else cpu). ? generated config diverges from checked-in (no
`relevance`/`tracknet` blocks, blur 100.0 vs 20.0).
84
85     ### 2.1 Segmenters (now under pipeline/segmenters) ?
86     - `@backend/pipeline/segmenters/base.py`
87     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. & `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
88     - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description MUST NOT touch `self.db`/`self.config`.
89     - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`) = the de-facto registration manifest.
90     - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`) ? WSL WASB substrate. Config `indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; hardcoded
relative out dir `output/wasb`; needs `{PROVIDER}_API_KEY` (missing -> []). Registered at module
bottom.
91     - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid (swaps runner; `detection_params` adds
`queue_length`). ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
92     - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`yolo_hybrid`, no YOLO remains ? torchvision detector + supervision ByteTrack).
`::PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::lazy_load_model`, `::extract_action_windows_from_chunk`,
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity computed at
`effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; trackers
per-chunk (windows merged by TIME not ID); `import torch` at module top.
93     - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded (ignores
`ai_max_workers`); `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates
analysis; `parse_time` accepts colon timestamps (warns).
94     - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
95
96     ### 2.2 Detectors & windowing (WSL-quarantined)
97     - `@backend/pipeline/detectors/__init__.py` ? docstring only (refs `docs/DECISIONS.md`
ADR-001). ? does NOT re-export runners; import submodules directly.
98     - `@backend/pipeline/detectors/tracknet_runner.py`
99     - `::TrackNetConfig` (+`.from_indexing_cfg`), `::TrackNetRunner` ? runs TrackNet
`predict.py` in WSL.
100     - `::to_wsl_mnt_path` (`C:\x`->`\mnt/c\x`), `::TrajectoryPoint`(frame, visible, x, y).
101     - `**::parse_trajectory_csv` @staticmethod** + `**::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi`; `visible==False`
resets prev (occlusion gap breaks track); `fps<=0`->`30`, `frame_width<=0`->`1280` silent;
`track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever).
102     - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (+`::WasbConfig`) ? stages
`wasb_infer.py` + video into WSL, runs in `wasb` env.
`::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)`. ?
`run_predict` does NOT pass cache flags (uses wasb_infer defaults flush=1000/batch=8/workers=4);
`wasb_infer.py` MUST be re-staged each run (editing Windows copy is inert until staged);
`wsl_bash` duplicated (drift risk).
103     - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. Reboot-resumable
detector cache: `::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.json'`,
`::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line

```

```

index`, rewrites to heal crash-truncated tail), `::manifest_compatible`,
`::atomic_write_text/_atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be np.array). ?
GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `runner.gpus=[0]` hardcoded
single-GPU; `extract_frames` cv2-PNG SLOW (prefer host ffmpeg).
104
105     ### 2.3 Eval & calibration (all pure; I/O lives in CLI drivers)
106     - `@backend/eval/metrics.py` ? scoring primitive. `::interval_iou`, `::match_intervals`
(greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`,
`::Match`, `::MatchResult`. ? greedy not Hungarian; `score()` indexes the SAME preds/gts used to
build `match_result` or boundary errors are silently wrong.
107     - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
108     - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?? labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement`, `::evaluate`. ? `generalization_gap
>=0.1` = overfit alarm.
109     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)` (?
hardcoded dup of real defaults), `::default_grid` (45 cfgs = 5x3x3), `::make_predict_fn`,
`::load_golden_gts` (? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ?
CLI defaults `fps=59.94/frame-width=1920` baked in.
110     - `@backend/eval/export_wasb_segments.py` ? trajectory+cfg -> golden/slicer CSV +
comparison. `::TUNED=(0.05,1.0,1.0)` (? hardcoded from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`. `--slice` requires `--video`.
111     - `@backend/eval/batch.py` ? corpus aggregation. `::aggregate` (micro pool TP/FP/FN, TP-
weighted mean_iou, macro F1), `::default_stages_for` (`auto`;
`::FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path`.
112     - `@backend/eval/harness.py` ? DB-pred scorer. `::STAGES=('candidates','final')`,
`::get_predictions` (candidates->`query_candidate_segments(detector=)`;
`final`->`query_play_segments({})` ALL segs, detector arg ignored),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate).
113     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? assumes `row['stats']` is already a dict ? the DB layer
deserializes it; a raw JSON STR would AttributeError, not zero-fill; missing fields default
0.0), `::label_candidates` (same IoU matching as evaluator),
`::build_training_set`->(X,y,intervals).
114     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
feature ORDER at predict must match fit (names are metadata only, not used in math); fixed 1000
epochs.
115     - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider`, `::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports
`annotation_registry` from `**`backend.annotations` NOT `backend.eval.annotations` (two
different "annotations"); gate fires on precision OR recall drop (F1 reported, not gated); first
run on a video silently passes (no baseline).
116
117     ### 2.4 Stitcher / tools / utils
118     - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. $ skipped tuple
`(idx,start,end,reason)`. ? if duration probe fails (0.0) NO boundary validation; `padded_start`
uses `begin_padding` (0.5)/`end_padding` (1.0) defaults; temp clips in a TemporaryDirectory under
output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`.
119     - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy,
kept in sync manually). `read_time` -> backend/eval/annotations.parse_timestamp.
120     - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe),
`::extract_video_chunk(...,reencode=False)`. ? uses `-t DURATION` (length, not end ts); copy
mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips.
121     - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (used by yolo_hybrid),
`::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.
122

```

```

123     ### 2.5 Validators ?
124     - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic), `::ValidationRegistry`, `::registry` (singleton). $ `validate(video_path, config,
context) -> ValidationResult`. ? **add a validator**: subclass +
`registry.register(MyValidator())` (registration order = execution order). ? validators share a
per-run `context` dict; exceptions wrapped as `passed=False` (never abort batch).
125     - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first).
126     - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (lowercased).
127     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails if format_check didn't run
first). `min_fps` default 29.9; min res 1280x720.
128     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? ? 10.0s keyframe-gap threshold hardcoded, comment says 8.0s (drift);
any backward jump fails.
129     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? ? `cut_threshold=0.6` hardcoded; samples at 2fps and caps at 100 SAMPLED
frames (~first 50s of video, not first 100 raw frames).
130     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold`(100.0) = fail; samples 15 frames.
131     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green; lazy `import backend.sports`; ? unknown sport -> empty sport cfg, falls back to
defaults, does NOT fail.
132
133     ### 2.6 Sports / Providers / Annotations / DB ?
134     - `@backend/sports/base.py::SportProfile` ABC
(`name`, `get_prompt`, `get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton). ?
**add a sport**: subclass `SportProfile`, `sport_registry.register(MyProfile)`. $ prompt emits
JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int),
shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS
timestamps; `get()` returns a fresh instance each call.
135     - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (upload->poll->generate JSON temp 0->delete);
`@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers
`gemini`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key
passed EXPLICITLY, unlike sport_registry); `get(name, api_key, model_name)`. $
`analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking `time.sleep(10)` poll; returns
`([], metrics)` on ANY failure (empty != error); lazy client (auth surfaces on first use).
136     - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::STATUSES`, `::AnnotationProviderRegistry`,
`::annotation_registry`. ? **add a tier**: subclass (define `name`), call
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py` (registry keys
off `provider_cls().name`). $ `fetch -> List[Interval]` sorted; contract says RAISE (not `[]`)
on unavailable (`[]` silently zeroes recall).
137     - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; -> `backend.eval.annotations.load_annotations`).
138     - `@backend/annotations/automated_provider.py::AutomatedProvider` (`auto`; injectable
`labeler`, `::warn_if_same_lineage` ORACLE RULE, default `_not_configured` RAISES
NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`.
139     - `@backend/annotations/__init__.py` ? ? `noqa F401` imports are load-bearing (removing
de-registers providers).
140     - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
(`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
`__init_db` (currently =1). Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_se
gment`, `link_candidate_to_segment`, `query_candidate_segments`, `query_play_segments`, `get_video`,
`clear_video_data`. ? every conn re-issues `PRAGMA foreign_keys=ON` (else CASCADE silently
skipped); `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest;
write helpers swallow exceptions (return False/None); events column is `type` not `event_type`;
`duration` stored not derived.

```

```

141
142 ---
143
144 ## 3. DATA CONTRACTS (canonical shapes ? one place)
145
146 $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
147 `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords. Row = `(fid, 1|0, x or -1, y or -1)`.
148 -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
149
150 $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`extract_action_windows_from_chunk`):
151 `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
152 `start/end = group_frame/fps + chunk_start`.
153
154 $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
155 `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
156 `detection_params` keys by detector:
wasb={detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}; tracknet
adds `queue_length`; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
157
158 $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
159 `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
160
161 $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
162
163 $ **GT / golden CSVs** (TWO conventions ? do not conflate):
164 - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
165 - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
166
167 $ **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
168 $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
169 $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},
...]}`; ? DB key = file MD5, manifest `video_id` is a label.
170
171 $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
172
173 $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
174
175 ---
176

```

```

177     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
178     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
179     - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
180     - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
181     - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
182     - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
183     - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
184
185     ---
186
187     ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
188     - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
189     - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
190     - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
191
192     ---
193
194     ## 6. GOTCHAS (cross-cutting footguns)
195     - **MD5 keying:** `video_id` = MD5 of WHOLE file everywhere ? but each entrypoint uses a
different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest `video_id` is a
label, NOT the DB key.
196     - **Three default segmenters:** config.json `gemini` vs setup.py `yolo_hybrid` vs
main.py `motion`. You may run a different detector than expected.
197     - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op for callers.
`analyze_video` also returns `[]` on real failure -> empty != "no rallies".
198     - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds if probing is skipped;
calibrate_wasb CLI bakes `59.94/1920`.
199     - **WSL `/mnt` can't see Google Drive / network mounts:** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
200     - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`.
201     - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
202     - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
203     - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless

```

```

`format_check` ran earlier (shared `context['ffprobe_meta']`).
204 - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately.
205 - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both.
206 - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). `regression.py` imports the latter.
207 - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
208 - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
209 - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat;
if probe fails (0.0) no boundary validation.
210
211 ---
212
213 ## 7. RAMP PATHS (to do X -> open Y::symbol)
214 | Task | Open |
215 |---|---|
216 | Tune candidate windowing (velocity/merge/min-dur) |
`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
217 | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in
`@backend/pipeline/segmenters/__init__.py` |
218 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
219 | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
220 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', cls)` |
221 | Add an annotation/GT tier | subclass
`@backend/annotations/base.py::AnnotationProvider`; register + import in
`@backend/annotations/__init__.py` |
222 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main`
(`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
223 | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
224 | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
225 | Score DB preds vs GT (one video) | `@run_eval.py::main` |
226 | Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
227 | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
228 | Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart; cache invalidates on
weights/sport/frames change |
229 | Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;
producers before consumers) |
230 | Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db`
migration ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
231 | Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
232
233 ---
234
235 ## 8. TEST MAP (`tests/test_*.py` -> subsystem)
236 | Test | Covers |
237 |---|---|
238 | `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
239 | `test_yolo_hybrid.py` / `test_tracknet_hybrid.py` | hybrid segmenters under

```

```

pipeline/segmenters (yolo torchvision+ByteTrack; tracknet 4-phase) |
240 | `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter`
(parse/validate/chunk) |
241 | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` parse_trajectory_csv
+ trajectory_to_action_windows (shared brain) |
242 | `test_wasb_cache.py` | `detectors.wasb_infer` resumable cache (serialize, contiguity
heal, manifest invalidation, atomic writes) |
243 | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
244 | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate |
245 | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict |
246 | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate |
247 | `test_calibration.py` | `eval.calibration` kfold/select_best/cross_validate/LOVO |
248 | `test_calibrate_wasb.py` | `eval.calibrate_wasb` CLI/run/grid |
249 | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment+comparison export
|
250 | `test_eval_training.py` | `eval.training` feature extraction + labeling |
251 | `test_eval_classifier.py` | `eval.classifier.LogisticRegression` fit/predict/save/load
|
252 | `test_eval_regression.py` | `eval.regression` baseline compare/gate |
253 | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
254 | `test_video.py` | `utils.video` duration/chunk |
255 | `test_geometry.py` | `utils.geometry` center/distance/velocity/IoU |
256 | `test_database.py` | `database.Database` schema/CRUD/migrations/version |
257 | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile |
258 | `test_providers.py` / `test_provider_gemini.py` / `test_gemini.py` | `providers`
registry + GeminiProvider lifecycle |
259 | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage) |
260 | (no dedicated test) | `pipeline/validators/*`, `pipeline/stitcher.compiler`,
`main.py`, `run*.py` orchestrators, `pipeline/detectors/wasb_runner`,
`pipeline/segmenters/wasb_hybrid`/`tracknet_hybrid` (covered indirectly) ?? exercised only
indirectly |
261

```

4. assistant (2026-06-07T09:17:12.392Z)

Now I have the full picture of the existing structure. Let me regenerate the complete CODE_MAP.md, applying the path migrations, new subsystems (config, reporting), refreshed test map, and corrected gotchas/ramp paths.

CODE_MAP.md

****Purpose:**** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

****Regenerate:**** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

****LEGEND****

- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `\$` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

1. PIPELINE

1A. Runtime: video file -> highlight reel

```
...
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
                                @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
-> video_id = MD5(whole file)      @backend/main.py::get_file_md5  (? chunk size varies
per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so
values agree)
-> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
    FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity ->
SceneCut -> Blur -> Relevance    (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
    [validation FAIL -> persist status="failed", return early; report STILL emitted in
finally]
-> db.add_video(video_id, filepath, status, [r.dict()])    @backend/infrastructure/database.py
-> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
-> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
-> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
    [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without
stitching]
-> finally: emit_indexer_report(...)    @backend/reporting/__init__.py (? ALWAYS runs ? even
on validation failure / exception; NEVER raises; grafts response["reports"])
-> select segment ids -> [(start,end)] (+ stitching padding)
-> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
    per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
...

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
...
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
-> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted via
db.add_candidate_segment]
-> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt) ? provider_registry
-> P4 db.add_play_segment + db.link_candidate_to_segment
...

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).
```

1B. WASB shuttle substrate (P2 of wasb_hybrid)

```
...
WasbHybridSegmenter.process_video                                @backend/pipeline/segmenters/wasb_hybrid.py
-> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
-> WasbRunner.run_predict(video, out_dir)
    stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
-> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector
pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
-> TrackNetRunner.parse_trajectory_csv(csv)    @backend/pipeline/detectors/tracknet_runner.py
(shared, re-exported by WasbRunner)
-> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
...

(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback
(30.0, 1280.0).)
```

1C. Calibration / eval flow (NO pipeline run unless told)

```
...
```

```

GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage ?
{candidates, final}
-> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
-> harness.evaluate -> EvalReport -> report_to_dict
    -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
    -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py [CI
gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}
@backend/eval/calibration.py
Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier)
@backend/eval/{calibrate_local,distill_local}.py
Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies}
@backend/eval/gemini_refine.py
Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
...

Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).

---
```

2. SUBSYSTEMS

2.0 Orchestration & config

```

- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI
mounts `/static` from `backend/api/static`.
- `::app` FastAPI; `::db_instance`; `::global_config` (Optional[AppConfig]) module globals
set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every
endpoint NPEs.
- `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:`
always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`.
`::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`.
`::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples  $\pm 3s$  vs
`indexing.motion_threshold` (dual model/dict read).
- `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
- ? `main()` runtime-override hack: `global_config.runtime_overrides = {"output_dir":
args.output_dir}` ? output dir lives ONLY on this dynamic attr (works because `AppConfig` is
`extra="allow"`), NOT in any config model; `db_path = os.path.join(args.output_dir,
output.db_name)`.
- ? `compile_highlights` reads `getattr(global_config.output, "output_dir", "output")` but
`OutputConfig` has **no `output_dir`** -> ALWAYS falls back to `"output"` (the
`runtime_overrides` value is NOT consulted here).
- ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks
automation), `default_segmenter` fallback `yolo_hybrid`, raw `json.load` config (bypasses
`backend.config`), `use_cache` defaults `True` (opposite of model/config `False`), skips
validation.
- `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`.
Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` reads
`config.json` via raw `json.load` (bypasses `backend.config`).
- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
`::load_config` is its OWN raw `json.load` dict loader; `segmenter_cls(db, cfg)` is called with
a **raw dict** cfg (not AppConfig).
```

```
- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::default_db_path` again raw `json.load`.

- `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config import load_config, AppConfig`.

- `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
  - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance); also what lets `main.py` graft `_runtime_overrides`.
  - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing", {}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
  - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)` .
  - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
  - `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
  - `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
  - `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`; ? NO `output_dir` field ? runtime-only).
  - `::StitchingConfig` (`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`).
  - `::Sport` Literal of 5 sports.
  - `::Config = AppConfig` alias.
- `@backend/config/loader.py` ? load/save with defaults-merge.
  - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
  - `::get_built_in_defaults` = `AppConfig().model_dump(...)` .
  - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
  - Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
  - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/--setup).
  - `::get_default_config` ? ? transitional plain-dict alias.
- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
  - **`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file wins at runtime.
- `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists).
  - `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu).
  - `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).
```

2.1 Segmenters (under pipeline/segmenters) ?

```
- `@backend/pipeline/segmenters/base.py`
  - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None, max_frames=None)` -> List[dict{id, start_time, end_time, duration, metadata}]`.
  - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is
```

```

masked).
- `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `wasb_hybrid`) ?
WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports
`WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
- `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `yolo_hybrid`, no
YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::PERSON_CLASS_ID=1` ?
(ultralitics used 0; wrong value -> zero detections). `::lazy_load_model` (lazy `torchvision`
import so module/tests load without it), `::extract_action_windows_from_chunk`,
`::make_tracker` (per-chunk ByteTrack ? track IDs chunk-local),
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at
`effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`;
pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI, no
P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ? pixel
absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.

```

2.2 Detectors & windowing (WSL-quarantined)

```

- `@backend/pipeline/detectors/tracknet_runner.py`
- `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field),
`::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
- `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame,visible,x,y).
- `**::parse_trajectory_csv` @staticmethod** + `**::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `''` -> `run_predict` returns None; predict.py only `mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
- `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (+`::WasbConfig`, from
`indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required
guard) ? stages `wasb_infer.py` (`::INFER_WIN`) + video into WSL, runs in `wasb` env.
`::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)`
(explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ?
`wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `wsl_bash`
duplicated (drift risk).
- `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo
modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text`/`atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker

```

STATEFUL -> always fully re-run; `--fresh` ignores cache.

- `@backend/pipeline/detectors/rally\_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate\_play\_axis` (larger-spread axis; ? ties -> 'x'), `::count\_net\_crossings` (sign changes vs median net; deadband jitter zone), `::gate\_windows`/`::apply\_gate` (`min\_crossings=2` default; kills single-toss service feeds).
- `::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by `calibrate_local` / `distill_local` / `export_wasb_segments`.

2.3 Eval & calibration (all pure core; I/O in CLI drivers)

- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval\_iou`, `::match\_intervals` (greedy, deterministic), `::score`->`::Score`(`.as\_dict()` = wire shape), `::pr\_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
- `@backend/eval/annotations.py` ? generic GT loader. `::load\_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse\_timestamp` (decimal or MM:SS/HH:MM:SS), `::write\_scaffold`.
- `@backend/eval/calibration.py` ? overfit guard. `::cross\_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave\_one\_video\_out` (honest cross-video; needs ?2 labelled videos), `::improvement\_report` (OPTIMISTIC ? selected on full GT), `::select\_best`, `::kfold\_indices`, `::pct\_improvement` (? from-zero -> `inf`), `::evaluate`. \$ `PredictFn` = Callable[[Config], List[Interval]] = the plug-in seam. ? `generalization\_gap >~0.1` = overfit alarm.
- `@backend/eval/calibrate\_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`, `::default\_grid` (45 cfs), `::make\_predict\_fn` (parses trajectory once, closes over `trajectory\_to\_action\_windows`), `::load\_golden\_gts` (\$`start\_time,end\_time` cols; ? SILENTLY skips bad rows ? opposite of `annotations.load_annotations`). ? `{load\_golden\_gts, make\_predict\_fn, BASELINE}` imported by 4 downstream drivers (`calibrate_local`, `export_wasb_segments`, `gemini_refine`, `audio/e1_probe`).
- `@backend/eval/calibrate\_local.py` ? distill Gemini->local thresholds (windowing + net-crossing gate, no cloud at runtime). `::local\_grid` (180 cfs), `::make\_local\_predict\_fn` (adds `rally\_gate.apply\_gate` when `min\_crossings>0`), `::build\_videos` (? `SystemExit` if a labels file yields no rallies), `::run`/`::main`. \$ `LocalConfig=(velocity,merge,min\_dur,min\_crossings)`, `BASELINE\_LOCAL=(0.02,2.0,2.0,0)`; manifest JSON shared with `distill_local`.
- `@backend/eval/export\_wasb\_segments.py` ? tuned cfg -> golden/slicer CSV + comparison + optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG\_FIELDS`, `::COMP\_FIELDS`, `::build\_comparison`, `::seconds\_to\_mms`, `::maybe\_slice` (lazy-imports `rally_slicer`). `--slice` requires `--video`. ? `write\_segments\_csv` output loads straight into `rally_slicer.load_rally_labels` AND re-imported by `gemini_refine` ? schema is load-bearing.
- `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in `run_phase3_eval`). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default\_stages\_for` ('auto'; `::\_FINAL\_ONLY\_SEGMENTERS={motion,gemini}`), `::resolve\_video\_path` (normalizes collector `./data/..` Windows paths), `::format\_aggregate`.
- `@backend/eval/harness.py` ? DB-pred scorer (no re-run). `::STAGES=('candidates','final')`, `::get\_predictions` (candidates->`query\_candidate\_segments(detector=)`); final->`query\_play\_segments({})`; ? unknown stage raises `ValueError`; reused by `regression.py`, `::evaluate`->`::EvalReport`(`::StageReport`)->`::report\_to\_dict` (input contract for `batch.aggregate`), `::format\_report\_text`.
- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO\_FEATURE\_NAMES` (5-d), `::extract\_yolo\_features` (? reads `row['stats']` dict from DB; missing fields default 0.0), `::label\_candidates` (same IoU `match\_intervals` as evaluator), `::build\_training\_set`->(X,y,intervals); `feature\_fn` is the substrate plug-in seam.
- `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn). `fit/predict\_proba/predict/to\_dict/from\_dict/save/load`. ? `class\_weight='balanced'` default; stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `sigmoid` clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `{"model":"logreg"}` tag.
- `@backend/eval/distill\_local.py` ? distill Gemini -> a LEARNED classifier (beyond `calibrate_local`'s threshold ceiling). `::FEATURE\_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build\_candidates` (recall-first proposal), `::predict\_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE\_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.
- `@backend/eval/gemini\_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; eval/tuning, NOT the live pipeline). `::merge\_overlaps(gap\_tol=0.0)` (reused

by `distill_local` with `gap_tol=1.0`), `::refine_window`` (? FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies``, `::run`/::main`` (`--full-video`` skips `--trajectory``). ? requires `GEMINI_API_KEY`` (raises `SystemExit`); imports `BASELINE`` from `calibrate_wasb + TUNED`/write_segments_csv`/seconds_to_mmss`` from `export_wasb_segments``.
- `@backend/eval/regression.py`` ? CI gate. `::DEFAULT_TOLERANCE=0.05``,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json``, `::regress``,
`::regress_with_provider`` (GT via ? `annotation_registry.get(tier)``;
`"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle`),
`::save_baseline`/::load_baselines``, `::RegressionResult``. ? imports `annotation_registry`` from
`**backend.annotations**` NOT `backend.eval.annotations``; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False`` (silent pass).

2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)

- `@backend/pipeline/audio/hit_detector.py`` ? classical-onset shuttle-hit detector. Pure numpy +
`stdlib`wave`` (NO `scipy`); `ffmpeg`` only for extraction. `::HitEvent``(t,strength),
`::extract_audio`` (`ffmpeg -> mono 16k PCM WAV`), `::load_wav``, `::onset_envelope`` (pure-numpy
STFT spectral flux; `band=(lo,hi)`` band-limit), `::detect_hits`` (adaptive `mean+k.std``;
`k`=sensitivity knob`), `::detect_hits_in_video``. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
- `@backend/eval/audio/e1_probe.py`` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits`` (groups thwacks; ? `min_hits>=2`` so a lone service-feed hit isn't a rally),
`::run`` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main``. reuses
`hit_detector`` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}`` + `metrics``.

2.4 Stitcher / tools / utils

- `@backend/pipeline/stitcher/compiler.py::VideoCompiler`` ? `compile_highlights(raw,`
`segments:[(start,end)], out_name)``. Per-clip re-encode -> `-f concat -c copy``. `::_parse_time``
(mirrors `gemini.parse_time``; ? unparseable -> 0.0), `::_get_video_duration``. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding``(0.5)/`end_padding``(1.0) defaults; temp clips
in a `TemporaryDirectory` under `output_dir``; raises `ValueError/FileNotFoundError/RuntimeError`; uses
`print()``. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
- `@backend/tools/rally_slicer.py`` ? standalone CLI. `::compute_clip_windows`` (pure, unit-
tested), `::load_rally_labels`` (golden schema; ? silently skips degenerate rows),
`::slice_rallies``, `::ClipWindow``(rally:str), `::BEGIN_PADDING=0.5`/::END_PADDING=1.0`` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp``.
- `@backend/utils/video.py`` ? `::get_video_duration`` (`ffprobe``; ? 0.0 on failure = "unknown"),
`::extract_video_chunk(...,reencode=False,scale_width=None)``. ? copy mode cuts at nearest
keyframe (drift) ? pass `reencode=True`` for short/accurate clips; `scale_width`` requires
`reencode=True``.
- `@backend/utils/geometry.py`` ? `::get_velocity_normalised`` (fraction of frame-width/s; ? raw-
px fallback if `width<=0``; used by `yolo_hybrid``), `::calculate_iou`` (? +1 px convention inflates
IOU), `::get_center/get_distance/get_velocity``.

2.5 Validators ?

- `@backend/pipeline/validators/base.py::VideoValidator`` ABC, `::ValidationResult`` (pydantic:
`validator_name,passed,message,details``, `::ValidationRegistry`` (ordered LIST ? registration
order = exec order; ? no dedup), `::registry`` (singleton). `$ validate(video_path, config,`
`context) -> ValidationResult``. `::run_all_with_context -> (results, context)`` (wraps each
validate in try/except -> failed result not abort); `::run_all`` discards context. ? `**add a`
`validator**`: subclass + `registry.register(MyValidator())``.
- `@backend/pipeline/validators/__init__.py`` ? registers in EXEC order: `FormatValidator``,
`ResolutionFPSValidator``, `PTSContinuityValidator``, `SceneCutDensityValidator``, `BlurValidator``,
`RelevanceValidator`` (? producers before consumers ? `format_check`` first; ? import mutates global
registry as side effect).
- `@backend/pipeline/validators/format.py::FormatValidator`` (`format_check``) ? `**PRODUCER**` of
`context['ffprobe_meta']``; ? MUST run before `resolution_fps``; container check is substring
`'mp4`` (so MOV-family passes); external `ffprobe``.
- `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator``
(`resolution_fps_check``) ? CONSUMES `ffprobe_meta`` (? hard-fails "Run format_check first" if
absent). `min_fps`` 29.9; min res 1280x720.
- `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator``
(`pts_continuity_check``) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded

(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES (too short).

- `@backend/pipeline/validators/scene\_cut.py::SceneCutDensityValidator` (`scene\_cut\_check`) ? OpenCV. ? `cut\_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps at 100 SAMPLED frames (~first 50s); `max\_scene\_cuts\_per\_minute` (0.5) is the config fail threshold.
- `@backend/pipeline/validators/blur.py::BlurValidator` (`blur\_check`) ? Laplacian var < `min\_blur\_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ? magic absolute, not resolution-normalized.
- `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance\_check`) ? HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`; 3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport -> empty cfg, falls back to defaults, does NOT fail.

2.6 Sports / Providers / Annotations ?

- `@backend/sports/base.py::SportProfile` ABC (`name`, `get\_prompt`, `get\_relevance\_config`);
- `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
- `@backend/sports/\_\_init\_\_.py::sport\_registry` (+`::SportRegistry`, auto-registers badminton; ? `register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`, `sport\_registry.register(MyProfile)`. \$ prompt emits JSON array `{start\_time(float DECIMAL SEC), end\_time, shuttle\_exchange\_count(int), shot\_count\_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS; `get()` returns a fresh instance each call.
- `@backend/providers/base.py::AIVideoProvider` ABC (`\_\_init\_\_(api\_key, model\_name)`);
- `@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX\_UPLOAD\_MB=500`; upload->poll->generate JSON temp 0->delete);
- `@backend/providers/\_\_init\_\_.py::provider\_registry` (+`::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass, `provider\_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api\_key, model\_name)`. \$ `analyze\_video(path, prompt, chunk\_duration=0.0, label='')` -> (segments:list, metrics:dict{upload_time, process_time, gen_time}). ? blocking 10s poll; returns `([], metrics)` on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible if process dies mid-run.
- `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
- `AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off `provider\_cls().name`, falls back to `\_\_name\_\_` if no-arg ctor throws; `list\_available()` -> LIST not dict), `::annotation\_registry`. ? **add a tier**: subclass (define `name`), `annotation\_registry.register(MyCls)` at module bottom + import in `\_\_init\_\_.py`. \$ `fetch` -> List[Interval] sorted; contract says RAISE (not `[]`) on unavailable.
- `@backend/annotations/file\_provider.py::FileProvider` (`file`; job_id IS the resolved CSV path; `\_resolve` -> `<csv\_dir>/<video\_ref>.csv`; -> `backend.eval.annotations.load\_annotations`; ? `guideline`/`sport` ignored).
- `@backend/annotations/automated\_provider.py::AutomatedProvider` (`auto`; GS-4 stub; injectable `::Labeler`, in-process `\_jobs` store, `::warn\_if\_same\_lineage` ORACLE RULE ? only warns, does not block; default `\_not\_configured` RAISES NotImplementedError). ? get via `annotation\_registry.get('auto', labeler=fn, lineage='...')`; registered as `auto` only because no-arg ctor succeeds.
- `@backend/annotations/\_\_init\_\_.py` ? ? `noqa F401` imports of file_provider/automated_provider are load-bearing (removing de-registers providers).

2.7 Infrastructure (DB) ?

- `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
- (`videos/play\_segments/events/candidate\_segments`), `PRAGMA user\_version` migration ladder in `::init\_db` (currently =1, `version<1` creates `candidate\_segments`). `::get\_connection` (? re-issues `PRAGMA foreign\_keys=ON` per connection ? else CASCADE/SET NULL silently skipped).
- Methods: `add\_video`, `add\_play\_segment`, `add\_event`, `add\_candidate\_segment`, `link\_candidate\_to\_segment`, `query\_candidate\_segments(detector=, only\_unlabeled=)`, `query\_play\_segments`, `get\_video`, `clear\_video\_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point (common fields columns, detector-specifics in `stats`/`detection\_params` JSON; index `idx\_candidates\_video\_detector`). ? `add\_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; events column is `type` not `event\_type`; `query\_play\_segments` filters use truthiness so `duration\_min=0`/empty-string = "no filter"; write helpers swallow exceptions.

2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)

- `@backend/reporting/\_\_init\_\_.py` ? `::emit_indexer_report(*, db, video_id, video_path, config,

```

val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False,
segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed config
(`config.model_dump(...)`) if `hasattr model_dump`); short-circuits to None if
unavailable/disabled; pulls play_segments/candidates from `db` (read-only
`query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to
`harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps
everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s
`finally:`); writes `

```

3. DATA CONTRACTS (canonical shapes ? one place)

```

$ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
`Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
-> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).

$ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
`{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.

$ **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
`{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL,
confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
`detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.

$ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.

```

\$ ****AI segment dict**** (provider output, consumed by hybrid P3 / gemini): `{start\_time, end\_time, shuttle\_exchange\_count?, shot\_count\_confidence?}` (offset by chunk/window start; `\_candidate\_id` stamped internally).

\$ ****GT / golden CSVs**** (TWO conventions ? do not conflate):

- generic (`annotations.load\_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.
- golden/slicer (`calibrate\_wasb.load\_golden\_gts`, `rally\_slicer.load\_rally\_labels`, `export.SEG\_FIELDS`): `rally\_number,start\_time,end\_time[,duration,start\_mmss,end\_mmss]`;

SILENTLY skips bad rows.

\$ ****det_raw.jsonl**** (one window/line): `{ "w":<window_index>,

"f":[[frame_id,[x,y,score,scale],...]], ...} }`. Lines MUST be contiguous (line N has `w==N`); first violation/JSONDecodeError -> truncate-heal.

\$ ****manifest.json**** (cache): `{version, frames\_dir(abspath), weights, sport, frames\_in, frames\_total, windows\_total, windows\_done}`. Reuse only if ALL match + `version==CACHE\_VERSION`.

\$ ****collector manifest.json**** (eval): `{eval\_sets:[{video\_id(label), local\_path, csv}, ...]}`; ? DB key = file MD5, manifest `video\_id` is a label.

\$ ****eval primitives****: `Interval=Tuple[float,float]` (start<end).

`Match{pred\_index,gt\_index,iou}`. `MatchResult{matches, unmatched\_pred\_indices(=FP), unmatched\_gt\_indices(=FN)}`. `Score{iou\_threshold,num\_pred,num\_gt,true\_positives,false\_positives,false\_negatives,precision,recall,f1,mean\_iou,mean\_start\_error,mean\_end\_error}.as\_dict()`.

\$ ****WSL detector candidate**** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]), 'score':float, 'scale':int }`; plain JSON form `[x,y,score,scale]`. ? `\_dets\_to\_tracker` MUST rebuild `xy` as np.array (tracker does `np.linalg.norm`).

\$ ****HitEvent**** (audio, `pipeline/audio/hit\_detector`): `{t:float(sec), strength:float}` ? onset-envelope peak.

\$ ****observability report**** (`reporting/harvest.record\_run` -> `RunRecorder`): stages

`validation/segmentation/ai\_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail}; flags `{code,faq\_ref}`; `video\_meta.fps\_source` ? {probed,fallback}. ? stage/metric/detail names are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:

`ai\_handoff.details.{api\_key\_present,ai\_based}`, `segmentation.metrics.segment\_count`, `segmentation.details.{segmenter\_actual,matches\_config\_default,segmenter\_explicitly\_requested,was\_reinges t}`, `validation.status`, `video\_meta.fps\_source`.

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims below are un

- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config\_name='eval', overrides=[dataset=<sport>, model=wasb, detector.model\_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::build_cfg.)

- `model=wasb` -> `name=hrnet`, `frames\_in=3`, `frames\_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)

- `detectors/detector.py::TracknetV2Detector.run\_tensor(imgs, affine\_mats)` -> (results, hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run\_tensor(imgs, trans)` returning `(batch\_results, \_)` ? consistent.)

- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame\_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build\_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)

- `dataloaders/dataset\_loader.py::ImageDataset(cfg, samples, input\_wh, output\_wh, transform, seq\_transform, is\_train)`; `samples=[{frames:[paths], annos:[{frame\_path, center:center(is\_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)

- Weights `pretrained\_weights/wasb\_badminton\_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.

5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified here)

- Indexer keys video by `**FILE MD5**`; collector keys by human ``video_id``. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
- ``python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@run_phase3_eval.py`.`
- Collector also emits golden rally labels (``rally_number,start_time,end_time,...``) -> consumed by ``@backend/eval/calibrate_wasb.py`` / ``export_wasb_segments.py`` / ``@backend/tools/rally_slicer.py``.

6. GOTCHAS (cross-cutting footguns)

- `**MD5 keying:**` ``video_id`` = MD5 of WHOLE file everywhere ? but each entriypoint uses a different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest ``video_id`` is a label, NOT the DB key.
- `**Two config-access idioms coexist:**` ``backend/main.py`` uses typed ``AppConfig`` (attribute access + ``getattr`` fallbacks); all four ``run_*.py`` scripts read ``config.json`` via raw ``json.load`` into a dict and bypass ``backend.config``. Default-drift between model / ``config.json`` / per-script literal fallbacks is the recurring footgun.
- `**Segmenter default defined in 5 places, 3 values:**` model ``yolo_hybrid``, ``config.json`` ``gemini``, ``main.py`` fallback ``motion``, phase3 default ``motion``, ``process_and_stitch`` fallback ``yolo_hybrid``. You may run a different detector than expected.
- `**`AppConfig.get` must return dict sections, not models:**` the legacy ``get("indexing",{ }).get(...)`` chains everywhere depend on it; the ``extra="allow"`` model also silently keeps typo'd config keys (no rejection).
- `**`output_dir` is runtime-only:**` lives on ``global_config._runtime_overrides`` (set in ``main()``), NOT in any config model. ``OutputConfig`` has no ``output_dir`` field, so ``compile_highlights`` always falls back to literal ``"output"``.
- `**Gemini free-tier / API-key dead path:**` hybrids + gemini return ``[]`` (not error) if ``{PROVIDER}_API_KEY`` missing OR WSL healthcheck fails ? a silent no-op. ``analyze_video`` also returns ``[]`` on real failure -> empty `!= "no rallies"`. The reporting ``silent_provider_noop`` / ``ai_empty_vs_no_rallies`` rules exist precisely to disambiguate this in the report.
- `**Reporting never breaks a run:**` ``emit_indexer_report`` swallows all exceptions (designed for ``main.py`` ``finally:``) and is emitted even on validation failure. ``obsreport`` is a SOFT dep ? absent -> no-op.
- `**spec.yaml <-> harvest.py coupling:**` rule ``path`` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.
- `**fps / frame_width must be PROBED not assumed:**` ``_probe_fps_width`` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; ``calibrate_wasb`` CLI bakes ``59.94/1920``. The report's ``fps_fallback_used`` flag surfaces this.
- `**WSL `/mnt` can't see Google Drive / network mounts:**` stage video into native WSL fs (``~/clips``) ? ``/mnt/c`` reads are slow and Drive-backed paths invisible.
- `**ffmpeg NOT in WSL**` by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. ``extract_frames`` uses cv2 PNG (slow) ? prefer host ffmpeg + ``--frames_dir``. ? Don't import ``wasb_infer`` from Windows code (it imports torch + WASB repo modules).
- `**GPU float nondeterminism:**` detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- `**`output/` is gitignored:**` DB, baselines, cache, reels live there. Predictions MUST exist in ``output/<db_name>`` before any eval/regression.
- `**Validator ordering coupling:**` exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; ``resolution_fps_check`` hard-fails unless ``format_check`` ran earlier (shared ``context['ffprobe_meta']``). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
- `**`add_video` INSERT OR REPLACE + FK CASCADE**` -> re-ingesting a ``video_id`` cascade-deletes its segments. Use ``clear_video_data`` deliberately. ``_get_connection`` MUST re-issue ``PRAGMA foreign_keys=ON`` per connection.
- `**Twin segmenters:**` ``wasb_hybrid.py`` / ``tracknet_hybrid.py`` are copy-paste; fixes to P3/P4

must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.

- **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
- **`calibrate\_wasb`** {load_golden_gts, make_predict_fn, BASELINE} is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini\_refine.merge\_overlaps` reused by `distill\_local`; `harness.get\_predictions` reused by `regression.regress`.
- **Hardcoded tuned constants:** `calibrate\_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
- **`timeout\_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
- **Audio NOT in the live pipeline:** `pipeline/audio/hit\_detector` + `eval/audio/e1\_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2x -> not adopted for fusion.

7. RAMP PATHS (to do X -> open Y::symbol)

```
| Task | Open |
|---|---|
| Change a config default / add a config field | `@backend/config/models.py` (single source of
truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
(default_segmenter) |
| Read config in new code | `from backend.config import load_config, AppConfig`
(`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
| Tune candidate windowing (velocity/merge/min-dur) |
`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
| Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in
`@backend/pipeline/segmenters/__init__.py`; if AI-based add to
`@backend/reporting/harvest.py::AI_SEGMENTERS` |
| Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
| Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)`
in `@backend/sports/__init__.py`; define prompt+HSV in new file |
| Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', Cls)` |
| Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`;
register + import in `@backend/annotations/__init__.py` |
| Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules` (declarative; ?
`when.path` must match harvest's stage/metric/detail names); facts in
`@backend/reporting/harvest.py` |
| Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage builders;
emit point is `@backend/main.py` `finally:` ->
`@backend/reporting/__init__.py::emit_indexer_report` |
| Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory
--labels --fps --frame-width`); trust `cv` recall not `fit` |
| Distill Gemini -> local (thresholds / learned) | `@backend/eval/calibrate_local.py::main`
(thresholds) OR `@backend/eval/distill_local.py::main` (learned classifier); both `--manifest` |
| Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main` (`--full-
video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
| Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main` (`--slice`
needs `--video`) |
| Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
| Score DB preds vs GT (one video) | `@run_eval.py::main` |
| Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing
DB; snapshot via `--update-baseline` |
| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
```

```
| Resume a crashed WASB pass | nothing to do ? `@backend/pipeline/detectors/wasb_infer.py::run`
auto-resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
| Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;
producers before consumers) |
| Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db` migration
ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
| Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
| Probe an audio signal (diagnostic) | `@backend/eval/audio/e1_probe.py::main`; detector in
`@backend/pipeline/audio/hit_detector.py` |
```

8. TEST MAP (`tests/test\_\*.py` -> subsystem)

All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked.

`@run\_all\_tests.py` (repo root) is the suite runner.

```
| Test | Covers |
|---|---|
| `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
| `test_yolo_hybrid.py` | `pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`
(torchvision+ByteTrack; `_PERSON_CLASS_ID` filter; candidate link; `store_yolo_candidates=False`
skip) |
| `test_tracknet_hybrid.py` | `pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter`
(4-phase; healthcheck-abort; `_probe_fps_width` fallback) |
| `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked path,
per-chunk offset, `shot_count`<=`shuttle_exchange_count`) |
| `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
| `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing count,
gate kills single-toss) |
| `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize, contiguity
heal, manifest invalidation, atomic writes, `default_cache_dir`) |
| `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
| `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate, `write_scaffold` |
| `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict; also
`@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
| `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate (micro/macro,
zero-div) |
| `test_calibration.py` | `eval.calibration`
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
| `test_calibrate_wasb.py` | `eval.calibrate_wasb` load_golden_gts/make_predict_fn/grid/run,
BASELINE |
| `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-one-
video-out |
| `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV schemas,
mmss; export->slicer reuse |
| `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
| `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
| `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
| `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies / learned-
vs-baseline |
| `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
| `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |
| `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label parse |
| `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
| `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU |
```

```

| `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
| `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig defaults
pinned, `extra` tolerance, `.get()` dict-compat) |
| `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`, `harvest.record_run`); ?
`importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-AppConfig +
`report.enabled=False` |
| `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref integrity
(every `faq_ref` resolves; actionable rules cite a faq_ref); ? `importorskip("obsreport")` |
| `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/run_all_with_context` -> `(results,context)`; exception captured not raised)
|
| `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains "decimal
seconds"/chunk value; relevance config keys) |
| `test_providers.py` / `test_provider_gemini.py` | `providers` registry + GeminiProvider
lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
| `test_annotation_providers.py` / `test_automated_provider.py` | `annotations` registry,
FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
| (no dedicated test) | individual `pipeline/validators/*` impls, `pipeline/stitcher.compiler`,
`main.py`, `run_process_and_stitch.py`, `pipeline/detectors/wasb_runner`,
`pipeline/segmenters/{wasb_hybrid,motion}`, `eval/audio/e1_probe` (diagnostic) ?? exercised
only indirectly |
| ? stale `.pyc` only | `tests/__pycache__/test_gemini_api.*` ? no live source exists |

```